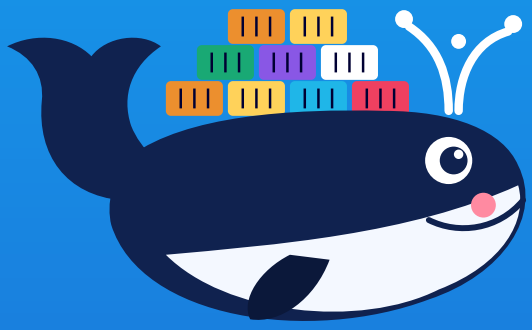
VISUAL STUDY NOTES

```
devops@docker: ~

devops@docker:~$ docker run -d -p 8080:80 nginx
9c2f1a0b2e3d
devops@docker:~$ docker ps
nginx   Up 3 seconds   8080->80/tcp
devops@docker:~$ docker images
nginx   latest   187MB
devops@docker:~$ docker stop 9c2f
9c2f
devops@docker:~$ _
```



Docker for DevOps

Containers, images and real-world practice, explained simply with visuals.

CREATED BY

in **Sharad Verma** @sharadverma638

Containers | Images | Dockerfile | Volumes | Networking | Compose | Scout

36 topics

60+ illustrations

100+ commands

</> **Want to learn DevOps?** ↗

github.com/sharadverma638/sv-opslibrary

Edition 2026 | Docker made easy for DevOps beginners | by Sharad Verma

Contents

36 sections. Read in order, or jump to what you need.

01 **What is Docker?** The problem, the toolbox and where it is used 2

02 **Containers vs Virtual Machines** Why containers start in a blink 4

03 **Docker Architecture** How the pieces fit and how isolation works 5

04 **Images, Containers and Registries** Recipe, dish and app store 7

05 **Installing Docker** Desktop, Engine and your first check 8

06 **The Docker CLI and Daemon** Commands, daemon.json and contexts 10

07 **Running Containers** Every common option, grouped 12

08 **Managing Containers** Start, stop, inspect, copy and restart 15

09 **Images and Layers** Tags, digests, save and load 18

10 **How Docker Stores Data** Where images and containers live 21

11 **The Dockerfile** Your first Dockerfile, line by line 23

12 **Dockerfile Instructions in Depth** Shell vs exec form, HEALTHCHECK, ARG 25

13 **Building and Optimizing Images** Cache, multi-stage, BuildKit, multi-platform 29

14 **Docker Init** Dockerfile and Compose in one command 33

15 **Port Mapping** Publish ports with -p 35

16 **Volumes and Data** Volumes, bind mounts and backups 37

17 **Networking Basics** Bridge networks, names and DNS 39

18 **Network Drivers and Legacy** bridge, host, overlay, macvlan and more 41

19 **Environment Variables and Config** -e, env files, ARG vs ENV 44

20 **YAML in Two Minutes** Maps, lists, quotes and common mistakes 46

21 **Docker Compose Basics** Services, file anatomy and core commands 48

22 **Compose File Reference** Line by line, with docker run equivalents 50

23 **Compose: depends_on and Health** Wait until a service is really ready 53

24 **Compose: Volumes, Networks, Vars** Top-level blocks, .env and secrets 55

25 **Compose Workflow and Commands** Every command plus a full example 57

26 **Docker Hub** Accounts, tokens, tags, limits and teams 60

27 **GitHub Container Registry** ghcr.io login, push and visibility 63

28 **Private and Other Registries** Registry options and registry:2 65

29 **Docker Scout** quickview, cves and recommendations 67

30 **Logging and Monitoring** docker logs, drivers, stats and events 69

31 **Troubleshooting and Cleanup** A checklist, common errors and pruning 71

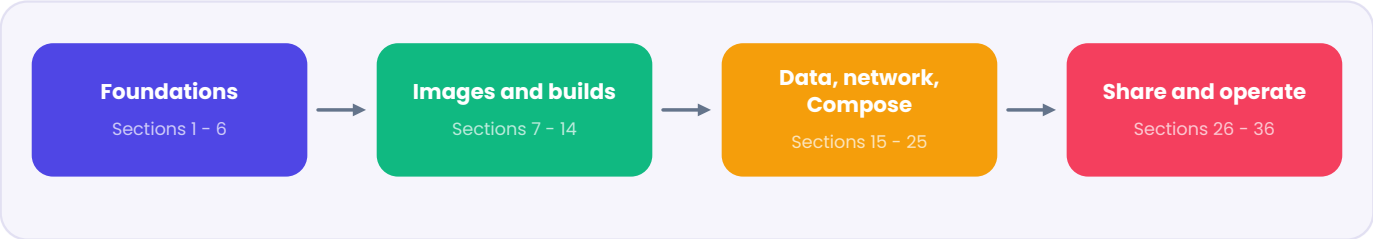
32 **Docker Security in Depth** Capabilities, rootless, trust and secrets 73

33 **Docker Swarm** Nodes, services, stacks and secrets 76

34 **Practical: Dockerize an App** A full Node.js walkthrough 78

35 **Practical: A Multi-container App** Everything together in one project 80

36 **Cheat Sheet and Glossary** Quick reference and key terms 82



Suggested study path

01 What is Docker?

Package once, run anywhere

Docker is a tool that packs an app **and everything it needs** (code, libraries, settings) into one box called a **container**. That box runs the same way on any computer.

The problem Docker solves

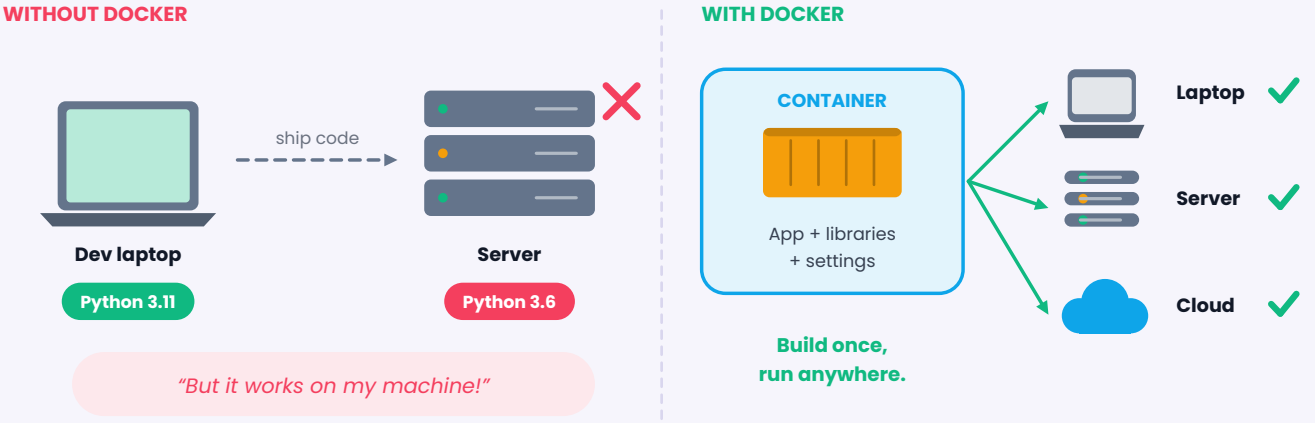


Fig 1. Without Docker the server can differ from your laptop. With Docker, the environment travels with the app.

THINK OF IT LIKE

A **shipping container**. It does not matter what is inside or which ship, truck or train carries it, the box fits everywhere. Docker does the same for software.

Why people use Docker

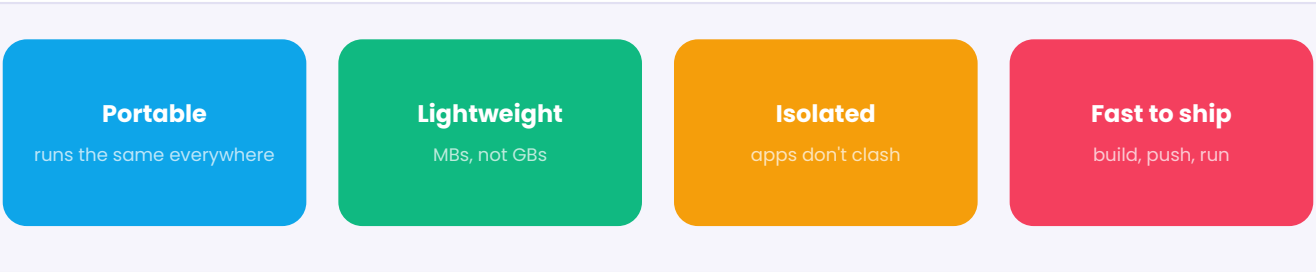


Fig 2. Four reasons Docker became the default way to ship software.

The Docker toolbox

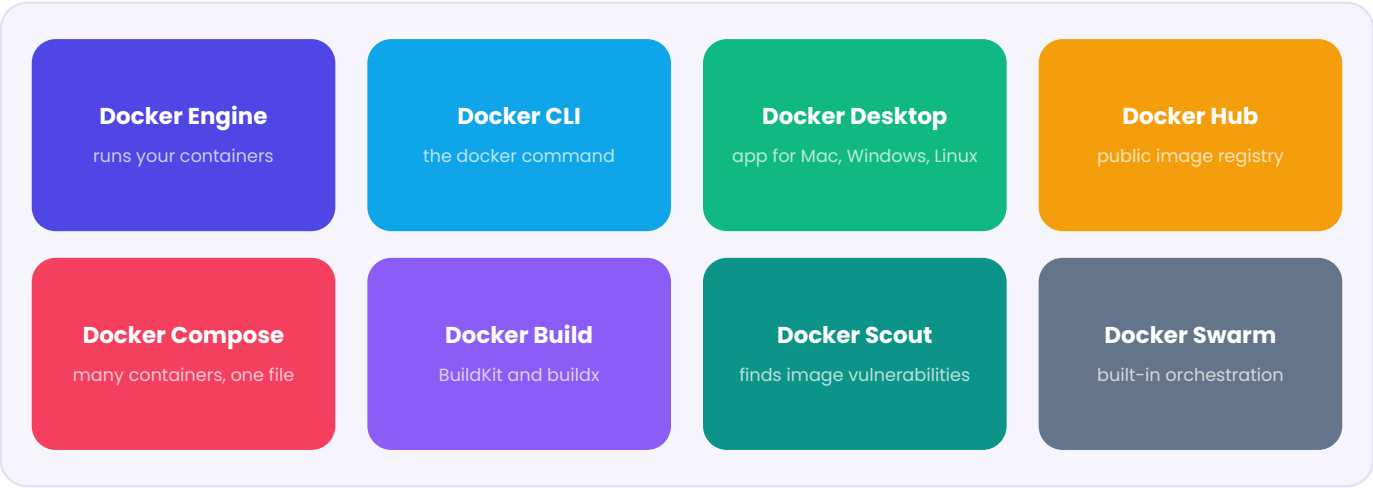


Fig 3. Docker is not one program. It is a family of tools that work together. You will meet each one in these notes.

Where you will see it

- **Development:** everyone on the team runs the exact same setup.
- **Testing:** start a clean copy of an app or database, then throw it away.
- **Microservices:** every small service gets its own container.
- **Trying software:** run a database in one command, delete it when done.
- **Learning:** practise Linux, databases and web servers without installing anything.

REAL EXAMPLE

Need MySQL for a project? Skip the installer. Run `docker run mysql`, use it, then remove it. Nothing is left on your machine.

02 Containers vs Virtual Machines

Light and fast vs heavy and isolated

Both let you run many isolated apps on one machine. A **virtual machine** copies a whole computer, including its own operating system. A **container shares** the host's operating system kernel.

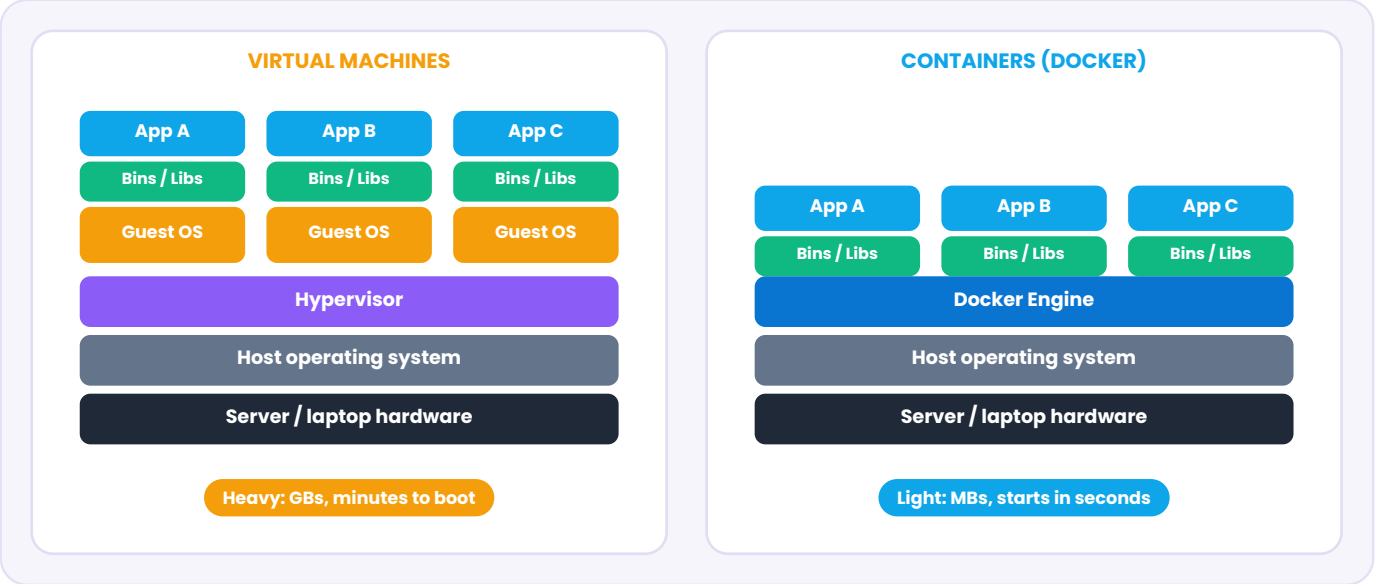


Fig 4. Each VM carries a full guest OS. Containers skip it, so they are tiny and start instantly.

	Virtual machine	Container
Size	Gigabytes	Megabytes
Start time	Minutes	Seconds or less
Operating system	Full guest OS for each VM	Shares the host kernel
Isolation	Very strong (hardware level)	Good (process level)
Best for	Different OSes, strict isolation	Many small apps, microservices

KEY IDEA

A container is just a **process with walls around it**. There is no second operating system to boot, which is why it starts in a blink.

NOTE

On Mac and Windows, Docker Desktop quietly runs **one small Linux VM** and puts your containers inside it, because containers need a Linux kernel.

03 Docker Architecture

Client, daemon, runtime and registry

Docker is **client-server**. You type commands into the **CLI**, the **daemon** does the real work, and a **registry** stores images.

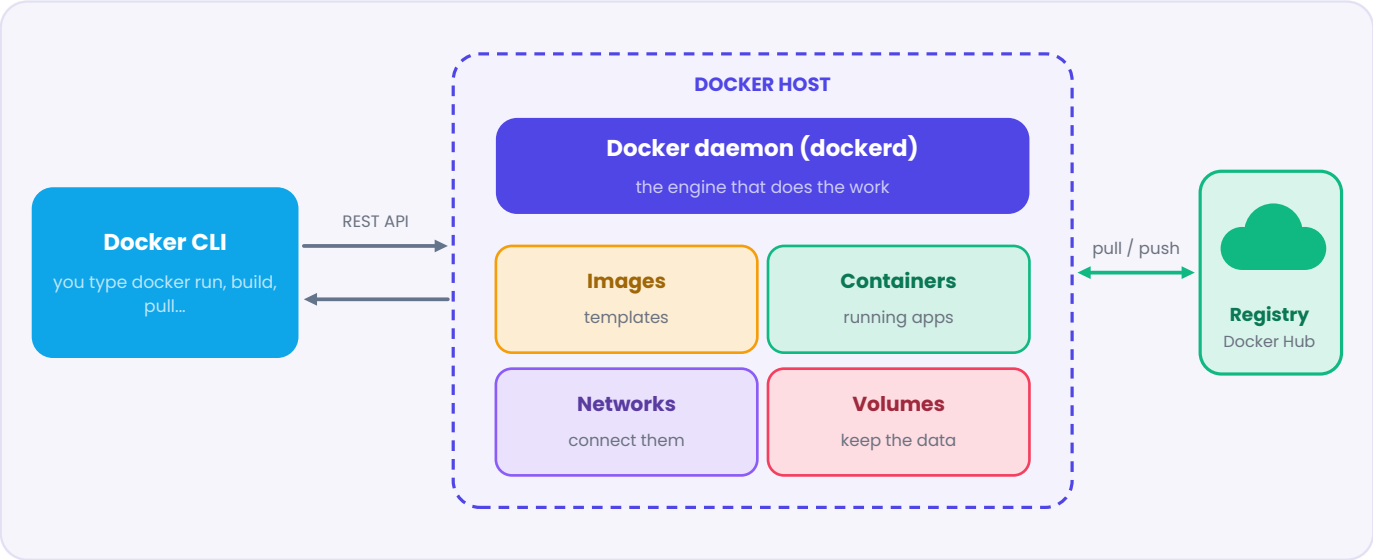


Fig 5. The CLI talks to the daemon over a REST API. The daemon pulls and pushes images to a registry.

Piece	What it does
Docker CLI	The `docker` command you type
Docker daemon	`dockerd`, builds and runs containers, manages networks and volumes
Images	Read-only templates used to create containers
Registry	Stores and shares images, for example Docker Hub

What runs your container

The daemon does not create containers by itself. It hands the job down a chain of small tools:

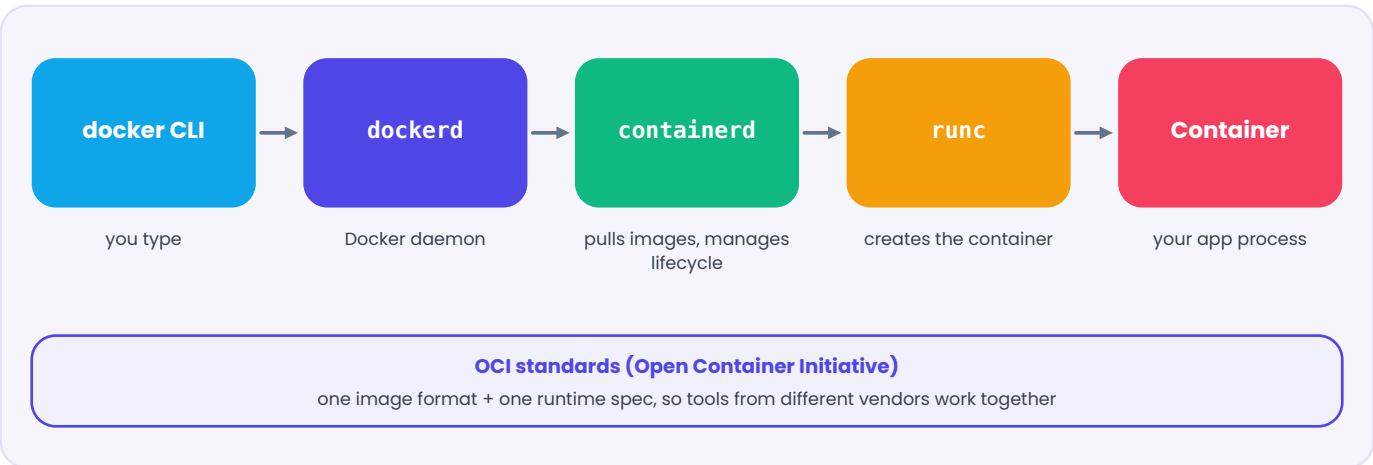


Fig 6. Each layer has one job. The same stack is used by other tools, which is why images are portable.

Component	Job
`dockerd`	The Docker daemon. Understands the Docker API, images, networks and volumes
`containerd`	Pulls images, stores them and manages container lifecycle. Also used by Kubernetes
`containerd-shim`	A tiny process per container, so containers keep running if the daemon restarts
`runc`	The low-level runtime that really creates the container using namespaces and cgroups

Under the hood: how isolation works

Docker does not invent isolation. It uses two built-in Linux features:

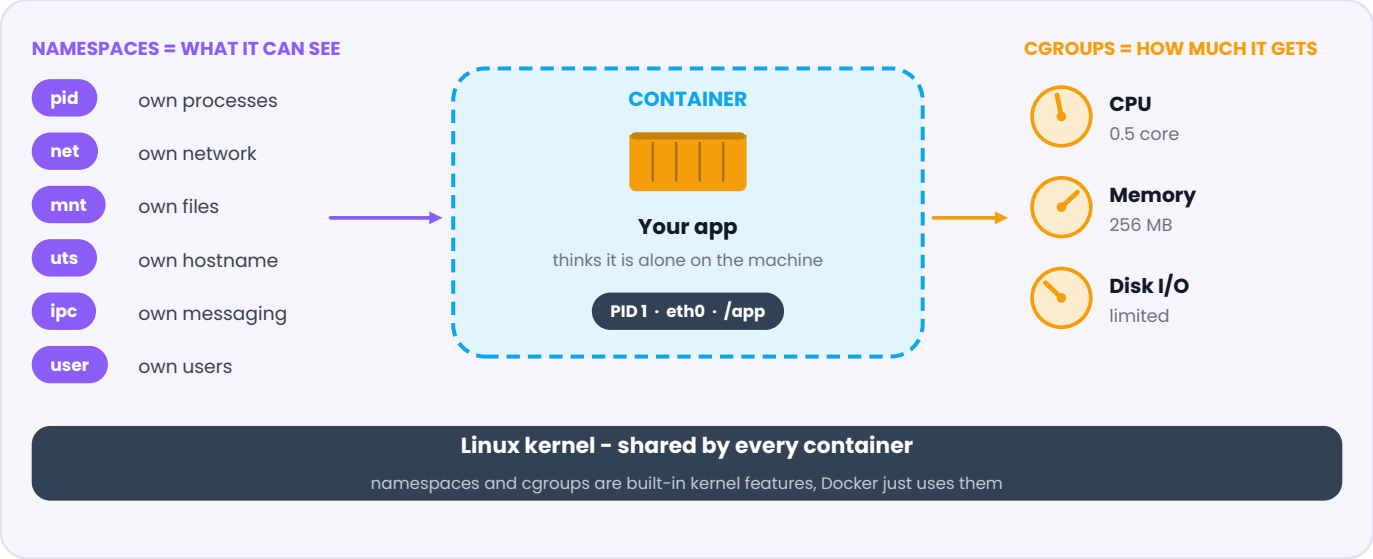


Fig 7. Namespaces give each container its own view. Cgroups cap how much CPU and memory it can use.

KEY IDEA

Namespaces decide what a container can **see**. **Cgroups** decide how much it can **use**.

04 Images, Containers and Registries

The three words you will hear most

An **image** is a frozen template. A **container** is a live copy started from it. A **registry** is where images are stored and shared.

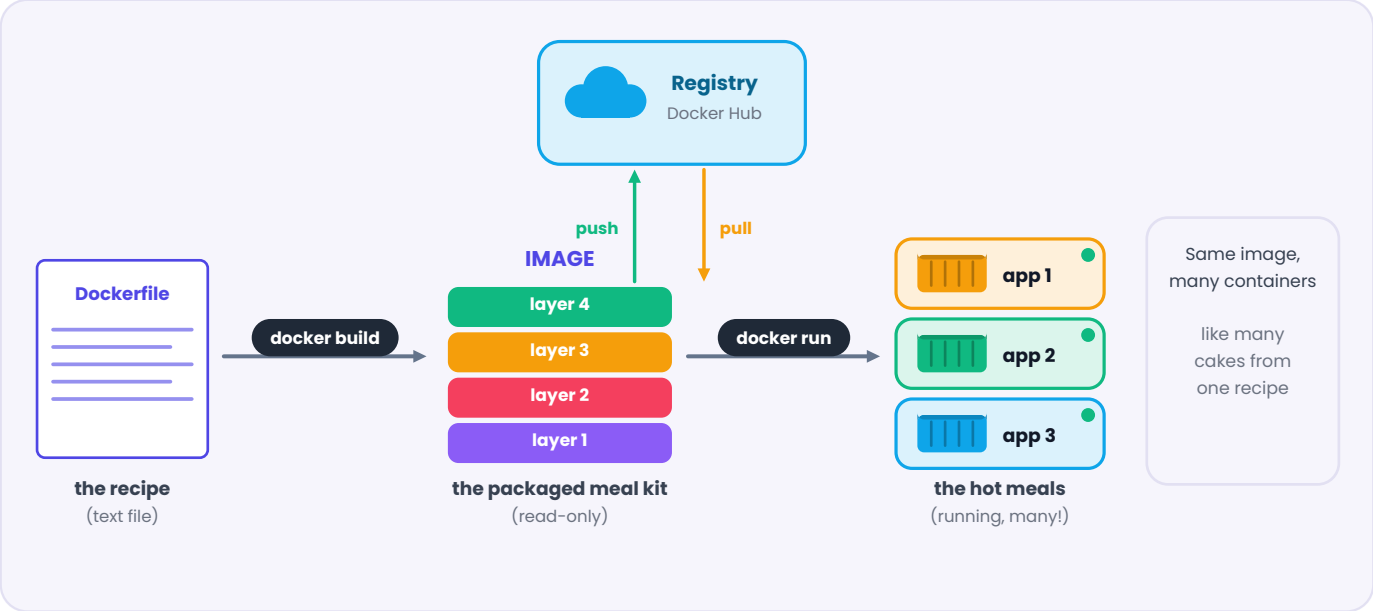


Fig 8. The big picture: write a Dockerfile, build an image, run containers from it, share it through a registry.

	Image	Container
What is it?	Read-only template (recipe + ingredients)	A running (or stopped) instance of an image
Can it change?	No, never	Yes, it has a thin writable layer
How many?	One image…	…can start many containers
Created with	`docker build`	`docker run`

THINK OF IT LIKE

An image is a **recipe**. A container is the **dish**. One recipe, as many dishes as you like.

NOTE

A **registry** works like an app store for images. **Docker Hub** is the biggest public one, and companies run private ones too.

05 Installing Docker

Pick the right edition for your system

Mac and Windows users install **Docker Desktop** (it has a friendly window). Linux servers install **Docker Engine** (just the engine and command line).

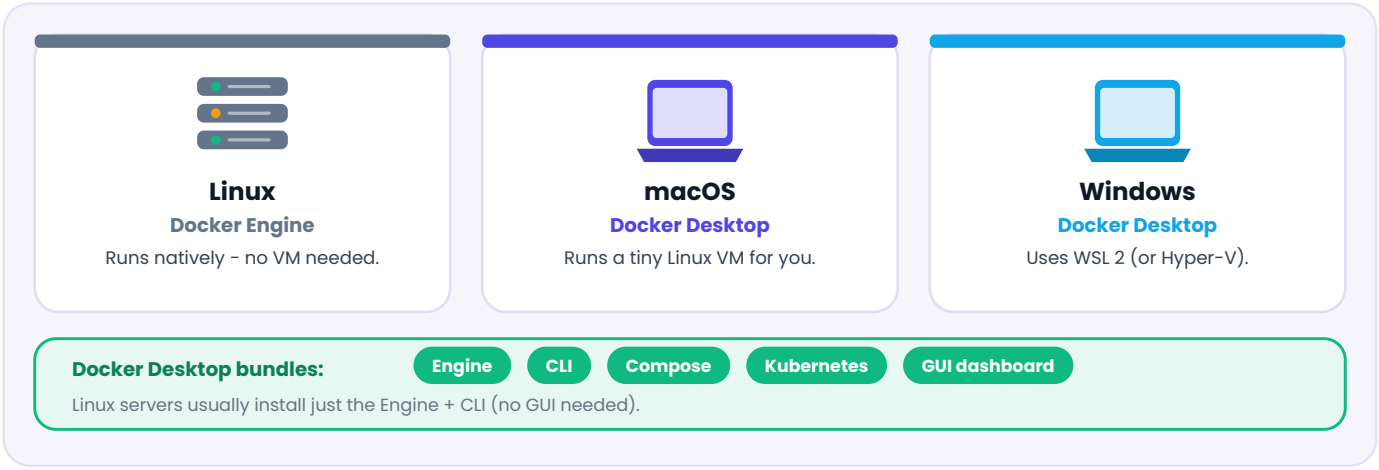


Fig 9. Which Docker to install, and how it runs on each system.

Quick install on Linux

```
terminal
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER    # run docker without sudo (log out and in again)
```

NOTE

The script is great for learning. On production servers, follow the apt or dnf repository steps on docs.docker.com/engine/install so updates come through your package manager.

Check that it works (you should see "Hello from Docker!")

```
terminal
docker --version           # the CLI version
docker compose version     # Compose comes with modern installs
docker run hello-world     # downloads a tiny image and runs it
```

Start Docker with the machine (Linux)

```
terminal
sudo systemctl enable --now docker     # start now and on every boot
sudo systemctl status docker           # is the daemon running?
```

Which one should I pick?

You are using	Install
A Windows or Mac laptop	Docker Desktop (pick the Apple chip or Intel download that matches your Mac)
A Linux desktop or server	Docker Engine, with the script above or your package manager
No install allowed	Play with Docker, a free browser lab at labs.play-with-docker.com

Docker Desktop settings worth knowing

Setting	Why it matters
Resources	How many CPUs, how much memory and disk the Docker VM may use
WSL 2 integration (Windows)	Lets your Linux distros use Docker directly
File sharing	Which folders can be bind-mounted into containers
Kubernetes	A one-click local Kubernetes cluster (optional)

NOTE

On **Windows**, turn on **WSL 2** (and CPU virtualization in the BIOS if asked). Docker Desktop guides you through it on first launch.

06 The Docker CLI and Daemon

How commands are organised and configured

Before running containers, learn how the **docker command is organised** and how to talk to the **daemon**.

How docker commands are built

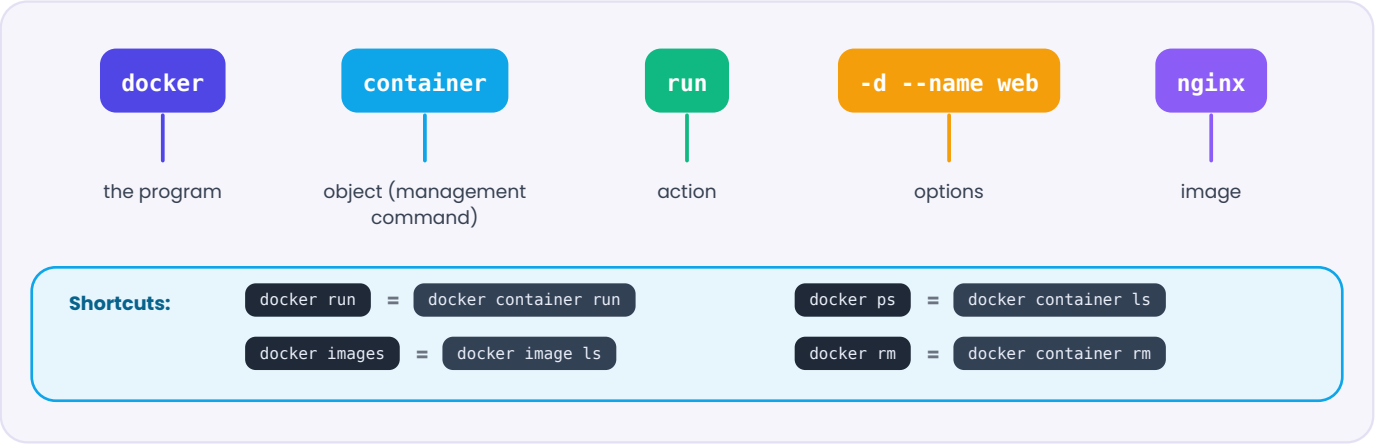


Fig 10. Commands follow docker object action. The old short forms still work and are what most people type.

The short forms (`docker run`, `docker ps`, `docker rm`, `docker images`) are what most people type. They still work and are used throughout these notes.

Get help and information

```
terminal
docker --help                # all commands
docker run --help            # every option of one command
docker version               # client AND server (daemon) versions
docker info                  # storage driver, runtime, containers, images...
```

Managing the daemon (Linux)

```
terminal
sudo systemctl start docker      # also: stop, restart, enable, disable
journalctl -u docker             # daemon logs
cat /etc/docker/daemon.json    # daemon settings (may not exist yet)
```

daemon.json: the daemon's settings file

```
/etc/docker/daemon.json
{
  "registry-mirrors": ["https://mirror.example.com"],
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "data-root": "/mnt/docker-data",
  "live-restore": true
}
```

Key	What it does
`registry-mirrors`	Pull through a mirror or cache instead of Docker Hub directly
`log-driver / log-opts`	Default logging driver and rotation for every container
`data-root`	Where Docker keeps images and volumes (default /var/lib/docker)
`insecure-registries`	Allow plain-HTTP registries (testing only)
`live-restore`	Keep containers running while the daemon restarts

TIP

After editing daemon.json run `sudo systemctl restart docker`. Without `live-restore`, restarting the daemon stops your containers.

Contexts: one CLI, many Docker hosts

A **context** tells the CLI which daemon to talk to. Switch contexts and the same commands run on another machine.

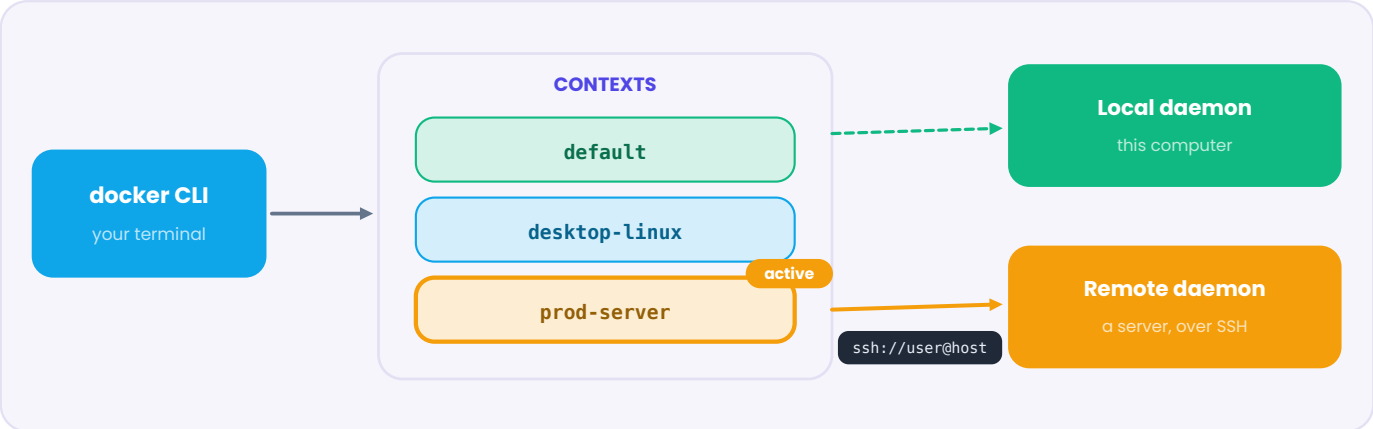


Fig 11. The active context decides where docker commands run.

```
terminal
docker context ls                                          # list contexts
docker context create prod --docker "host=ssh://user@server"   # remote over SSH
docker context use prod                                    # now docker talks to the server
docker --context default ps                                # one-off command on another context
```

WATCH OUT

Anyone who can reach `/var/run/docker.sock` effectively has **root on that machine**. Never share that socket carelessly.

07 Running Containers

docker run, explained

`docker run` **creates and starts** a container from an image. Read the command left to right:

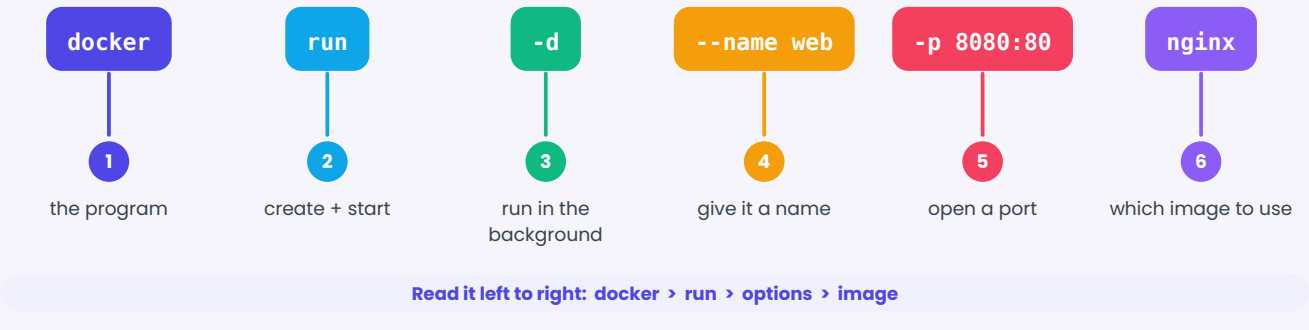


Fig 12. Every docker run command follows the same shape: program, action, options, image.

What happens when you press Enter



Fig 13. Docker only downloads the image if it is not already on your machine.

Options, grouped by purpose

The basics

`-d` run in the background	`-it` interactive terminal	`--name web` friendly name
`--rm` delete when it stops	`--pull always` always check for a newer image	`--init` tiny init as PID 1

Networking

`-p 8080:80` publish a port	`-P` publish all EXPOSEd ports	`--network my-net` join a network
`--hostname app1` set the hostname	`--add-host a:1.2.3.4` add a hosts entry	`--dns 8.8.8.8` custom DNS server

Storage and environment

`-v data:/path` volume or bind mount	`--mount type=...` explicit mount syntax
`--tmpfs /tmp` memory-only folder	`-e KEY=value` environment variable
`--env-file .env` variables from a file	`-w /app` working directory

Behaviour and limits

`--restart always` restart policy	`--memory 256m` RAM limit
`--cpus 0.5` CPU limit	`-u 1000:1000` run as this user
`--read-only` read-only filesystem	`--entrypoint sh` replace ENTRYPOINT

Replace the default command

```
terminal
docker run ubuntu echo "hi"       # runs echo instead of the image's default CMD
docker run alpine sleep 5         # any command the image contains
docker create --name web nginx    # create only (not started yet)
docker start web                  # start it later
```

Try it: your first web server

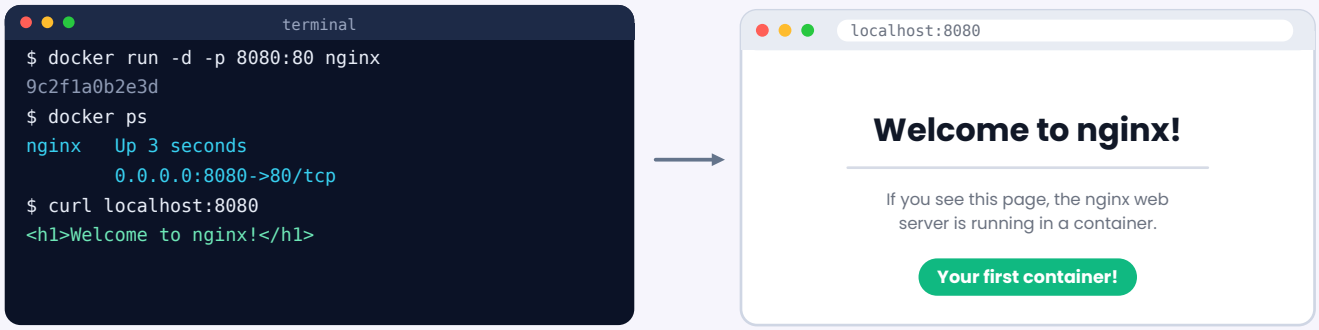


Fig 14. One command starts nginx. Open localhost:8080 and the page is served by your container.

Foreground or background?

By default a container **takes over your terminal**. Add `-d` to run it in the background.

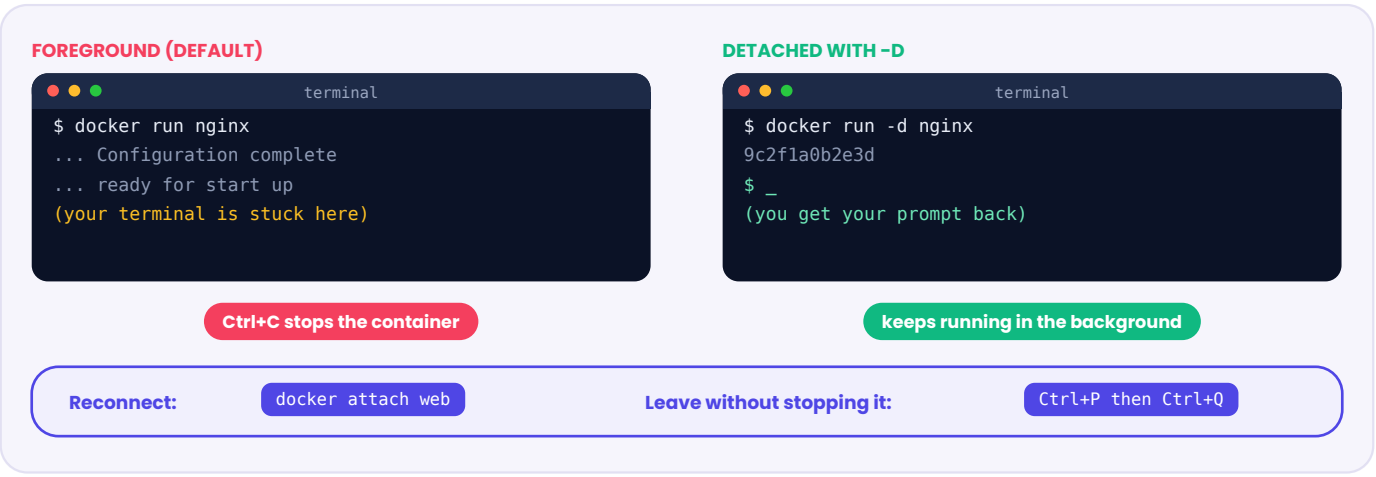


Fig 15. Use -d for servers. Attach later if you want to look inside.

Talk to a container

Add `-it` to get a terminal **inside** a container. Here is a throwaway Ubuntu:

```
terminal
docker run -it --rm ubuntu bash       # you are now inside the container
cat /etc/os-release                      # it thinks it is Ubuntu
exit                                     # leave (the container stops and is removed)
```

KEY IDEA

A container **lives as long as its main process**. When `bash` exits, the container stops. A web server keeps running, so its container stays up.

08 Managing Containers

Lifecycle, signals and restart policies

A container moves through a few **states**. These commands move it between them and let you look inside.

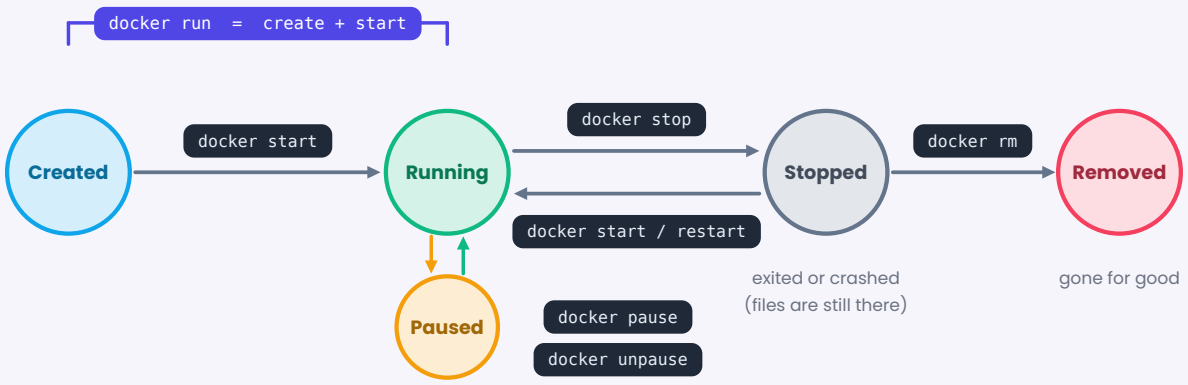


Fig 16. The life of a container. A stopped container still exists until you remove it.

Everyday commands

`docker ps` running containers	`docker ps -a` all, even stopped	`docker logs web` see its output
`docker stop web` stop it politely	`docker start web` start it again	`docker rm web` delete it
`docker exec -it web sh` open a shell inside	`docker inspect web` full details (JSON)	`docker stats` live CPU and memory

More container commands

`docker restart web` stop, then start	`docker pause web` freeze all processes	`docker unpause web` unfreeze it
`docker kill web` stop immediately	`docker rename web w2` change the name	`docker cp web:/f ./` copy files in or out
`docker diff web` files changed inside	`docker top web` processes inside	`docker port web` show port mappings
`docker wait web` wait until it stops	`docker attach web` reconnect to it	`docker update --memory 512m web` change limits live

```
terminal
docker cp web:/etc/nginx/nginx.conf ./nginx.conf     # copy a file OUT of a container
docker cp ./index.html web:/usr/share/nginx/html/     # copy a file IN
docker exec -it -u root -w /app web sh                 # shell as root, starting in /app
docker exec web env                                    # run one command, then return
```

How a container stops

`docker stop` is polite first, forceful later:



Fig 17. Handle SIGTERM in your app so it can save work before it exits.

Exit code	Meaning
0	Finished successfully
1	The app failed (a generic error)
125 / 126 / 127	Docker error / command cannot run / command not found
137	Killed (SIGKILL): out of memory, docker kill, or stop timed out
139	Crashed with a segmentation fault
143	Stopped by SIGTERM (a normal docker stop)

TIP

See why a container exited: `docker ps -a` shows the exit code, and `docker inspect web --format '{{.State.ExitCode}}'` prints it alone.

Restart policies

Tell Docker what to do when a container stops or the machine reboots:



Fig 18. Use unless-stopped or always for services that must stay up.

Make an image from a container

```
terminal
docker commit web myimage:v1       # snapshot a container as a new image
docker export web -o web.tar         # the container's filesystem as a tar file
```

WATCH OUT

`docker rm` deletes the container **and** anything it wrote that was not saved in a volume. And prefer a Dockerfile over `docker commit`, because a Dockerfile can be repeated and reviewed.

09 Images and Layers

How images are built, named and moved

An image is a **stack of layers**. Layers are cached and shared, so images stay small and pulls are fast.

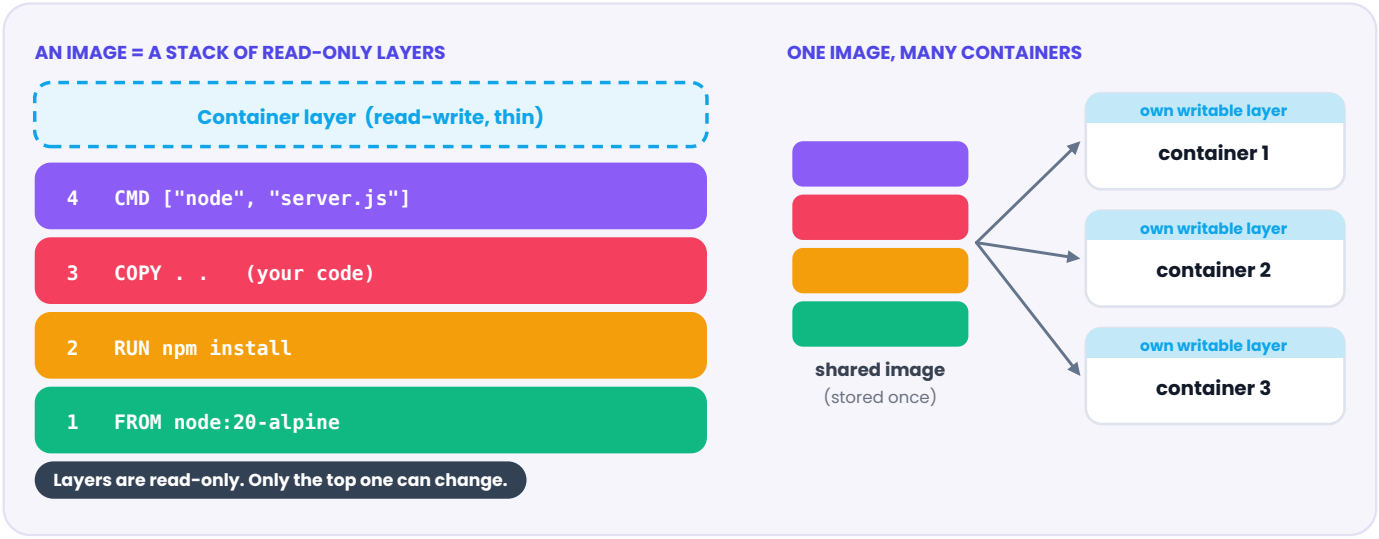


Fig 19. Each instruction adds a read-only layer. A running container adds one thin writable layer on top.

Image names

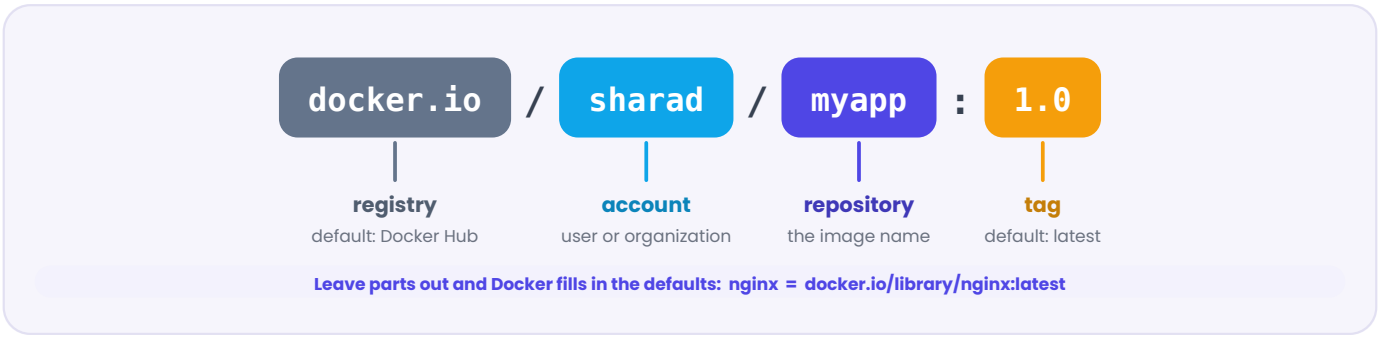


Fig 20. A full image name has four parts. Docker fills in the defaults for anything you leave out.

Tags and digests

A **tag** is a friendly label that the owner can move. A **digest** is a fingerprint of the exact content, so it can never change.

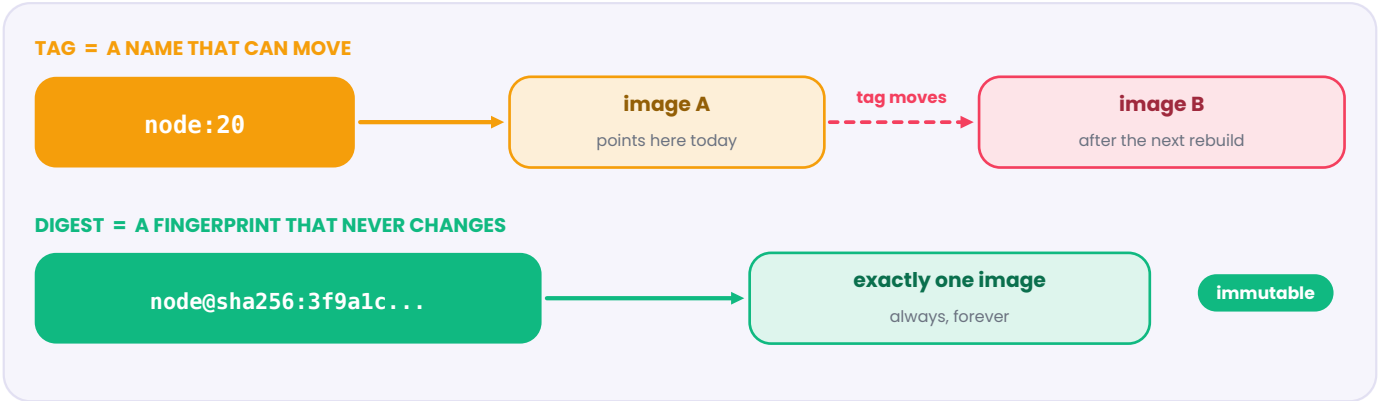


Fig 21. Pin a digest when you need to be certain which image you run.

```
terminal
docker pull node:20                          # by tag
docker pull node@sha256:3f9a1c...             # by digest (exact content)
docker images --digests                       # show digests of local images
```

Image commands

`docker pull nginx:1.27` download an image	`docker images` list local images	`docker rmi nginx` delete an image
`docker tag a b` add another name	`docker history nginx` show its layers	`docker image inspect nginx` full details (JSON)
`docker search nginx` search Docker Hub	`docker push user/app` upload to a registry	`docker image prune` remove dangling images
`docker pull --platform linux/arm64` pick a platform	`docker images -q` only the image IDs	`docker images -a` include hidden layers

Copy-on-write

Image layers are read-only. When a container changes a file, Docker **copies** it up to the writable layer first.

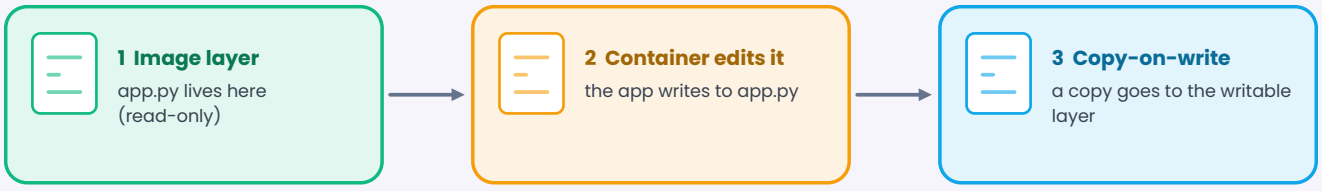


Fig 22. Copy-on-write keeps the original image clean for every other container.

See the layers yourself

```
terminal
docker history myapp:1.0          # example output:
IMAGE          CREATED BY                              SIZE
a1b2c3d4e5f6   CMD ["node" "server.js"]                 0B
b2c3d4e5f6a1   COPY . .                                 2MB
c3d4e5f6a1b2   RUN npm install                         38MB
d4e5f6a1b2c3   FROM node:20-alpine (base)               135MB
```

TIP

A **dangling** image has no name or tag (it shows as `<none>`). It is usually a leftover from a rebuild. `docker image prune` removes them.

Move images without a registry

Need to copy an image to a machine with no internet? Save it to a file and load it there.

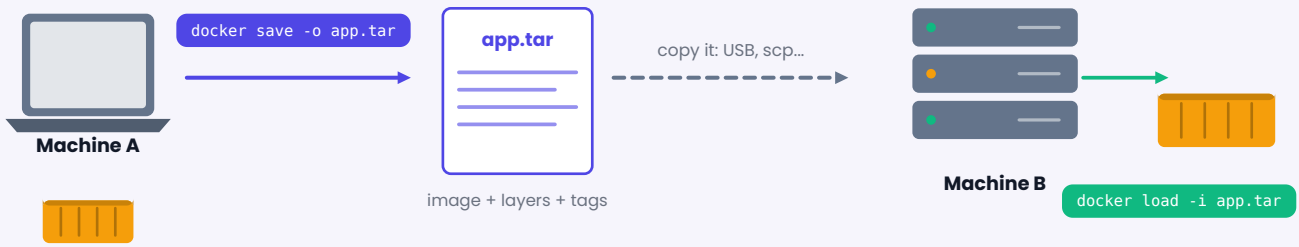


Fig 23. save and load keep layers and tags. Great for offline machines.

Pair	Works on	Keeps
`docker save / load`	images	all layers, tags and history
`docker export / import`	containers	just the flattened filesystem, no history or settings

WATCH OUT

`:latest` does **not** mean newest. It is just a tag name. In real projects, pin a version like `node:20.11` or a digest.

10 How Docker Stores Data

Storage drivers and overlay2

Docker stores images and containers on disk using a **storage driver**. Knowing how helps you understand image size, speed and where your data really lives.

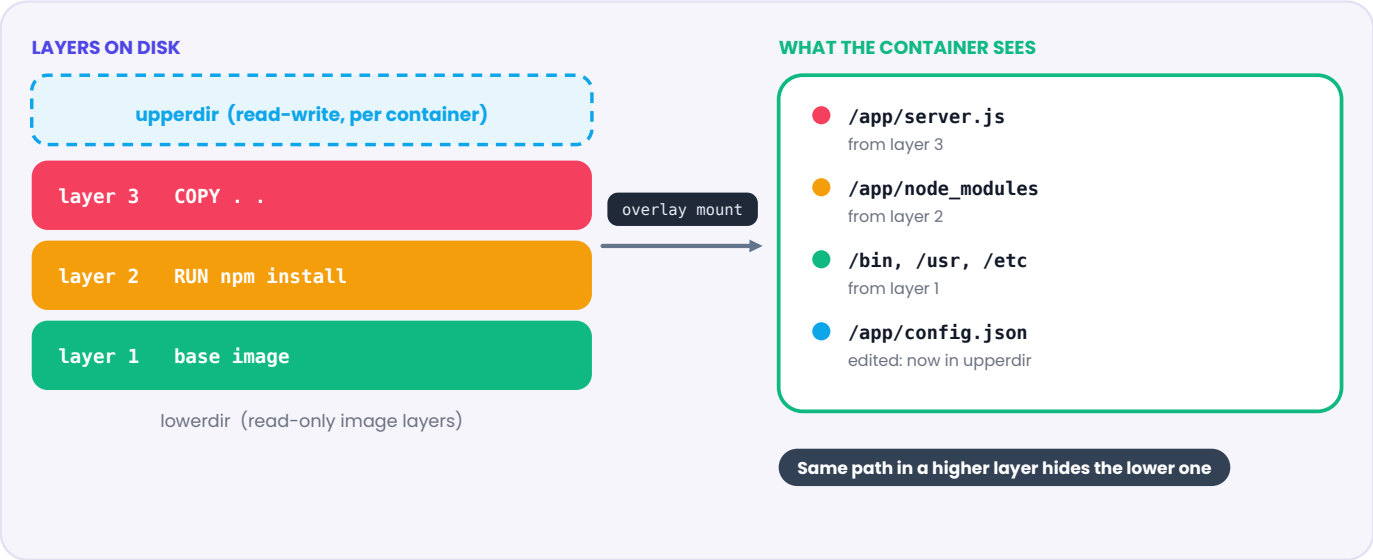


Fig 24. overlay2 stacks image layers read-only and puts a writable layer on top. The container sees them merged as one filesystem.

Where Docker keeps things (Linux)

Folder inside /var/lib/docker	What is there
`overlay2/`	Image layers and container writable layers
`volumes/`	Named volumes (your persistent data)
`containers/`	Per-container config and log files
`image/`	Image metadata and layer database
`network/`	Network settings

Storage drivers

Driver	Status	Notes
`overlay2`	Default, recommended	Fast and efficient on modern Linux filesystems
`fuse-overlayfs`	Rootless mode	Used when running Docker without root
`btrfs / zfs`	Supported	Use when the host already runs that filesystem
`vfs`	Testing only	No sharing between layers, very slow and large
`aufs, devicemapper, overlay (v1)`	Outdated	Deprecated or removed in recent releases. Use overlay2

```terminal
docker info | grep "Storage Driver"      # which driver you use
docker system df                         # space used by images, containers, volumes, cache
docker system df -v                      # the same, item by item
```

NOTE

Recent Docker versions can also store images using containerd's image store instead of the classic one. The commands you type stay the same. `docker info` shows which one is active.

KEY IDEA

Anything a container writes lives in its thin writable layer and **disappears with the container**. Persistent data belongs in a **volume**.

11 The Dockerfile

A recipe for your image

A Dockerfile is a **plain text file of instructions**. `docker build` reads it from top to bottom and produces an image.

Your first Dockerfile

```
Dockerfile
FROM alpine
CMD ["echo", "Hello from my first image"]
```

```
terminal
docker build -t hello .          # build an image called hello
docker run hello                 # prints: Hello from my first image
```

A real Dockerfile, line by line

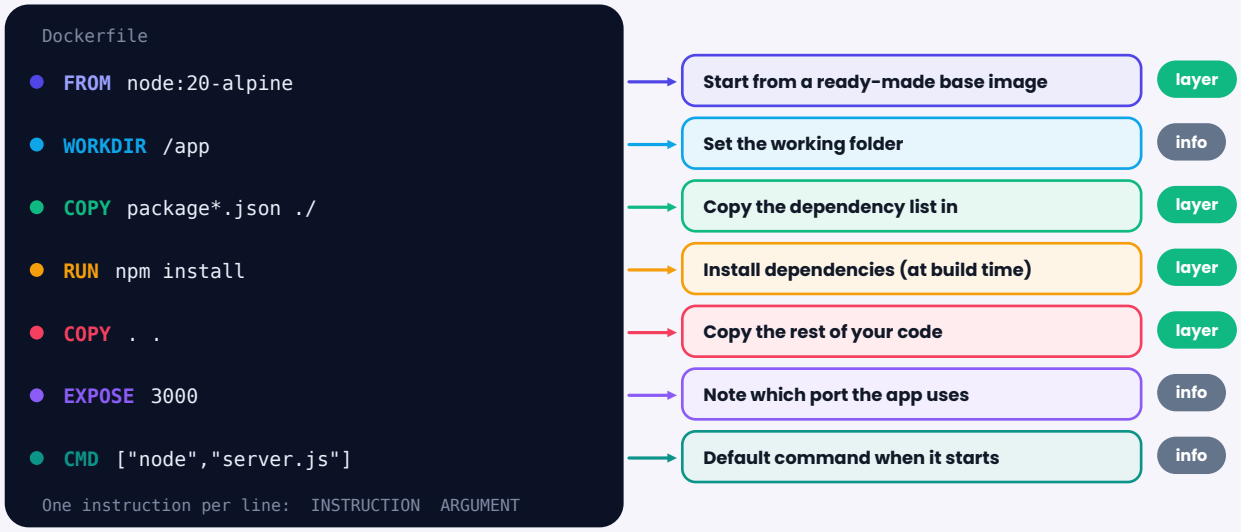


Fig 25. Instructions that change files each create a layer. The rest only store settings.

KEY IDEA

`RUN` happens when you **build** the image. `CMD` happens when you **start** a container.

Five questions that write the Dockerfile for you

Ask yourself	Instruction
Which base image should I start from?	`FROM node:20-alpine`
Where should my files live?	`WORKDIR /app`
What files go inside?	`COPY . .`
What must be installed?	`RUN npm install`
Which port does it use?	`EXPOSE 3000`
How does it start?	`CMD ["node", "server.js"]`

Ground rules

- One instruction per line. Lines starting with `#` are comments.
- Instructions are not case sensitive, but **UPPERCASE** is the habit that makes files readable.
- Every Dockerfile starts with `FROM` (only `ARG` may come before it).
- Each instruction that changes the filesystem adds one **layer**.
- The file is named `Dockerfile`. Use `-f` to build a different file: `docker build -f Dockerfile.dev .`

12 Dockerfile Instructions in Depth

Every instruction, with examples

Every Dockerfile instruction, what it does, and when to use it.

Core instructions

Instruction	What it does	Example
`FROM`	Base image. Can be named for multi-stage builds	`FROM node:20 AS build`
`WORKDIR`	Set (and create) the working folder	`WORKDIR /app`
`COPY`	Copy files from your project into the image	`COPY src/ ./src/`
`RUN`	Run a command at build time (adds a layer)	`RUN npm ci`
`ENV`	Set an environment variable that stays in the image	`ENV PORT=3000`
`EXPOSE`	Document the port the app listens on	`EXPOSE 3000`
`CMD`	Default command when the container starts	`CMD ["node", "app.js"]`
`ENTRYPOINT`	The fixed program. CMD supplies its default arguments	`ENTRYPOINT ["node"]`

More instructions

Instruction	What it does	Example
`ARG`	Build-time variable (gone at run time)	`ARG NODE_VERSION=20`
`ADD`	Like COPY, plus URLs and auto-unpacking of .tar files	`ADD app.tar.gz /app`
`USER`	Run the next steps and the container as this user	`USER node`
`VOLUME`	Mark a folder as persistent data	`VOLUME /var/lib/data`
`LABEL`	Add metadata (key=value)	`LABEL version="1.0"`
`HEALTHCHECK`	Teach Docker how to test the app	`HEALTHCHECK CMD curl -f ...`
`STOPSIGNAL`	Signal sent by docker stop	`STOPSIGNAL SIGINT`
`SHELL`	Change the shell used by shell-form commands	`SHELL ["/bin/bash", "-c"]`
`ONBUILD`	Run an instruction when this image is used as a base	`ONBUILD COPY . /app`

COPY or ADD?

	COPY	ADD
Copies local files	Yes	Yes
Downloads a URL	No	Yes (prefer curl in a RUN)
Unpacks .tar archives	No	Yes, automatically
Advice	Use this by default	Only when you need unpacking

```
Dockerfile
COPY --chown=node:node . /app        # set the owner while copying
COPY --from=build /app/dist ./dist     # copy from an earlier build stage
```

Shell form or exec form?

`RUN`, `CMD` and `ENTRYPOINT` can be written two ways. They are not the same.

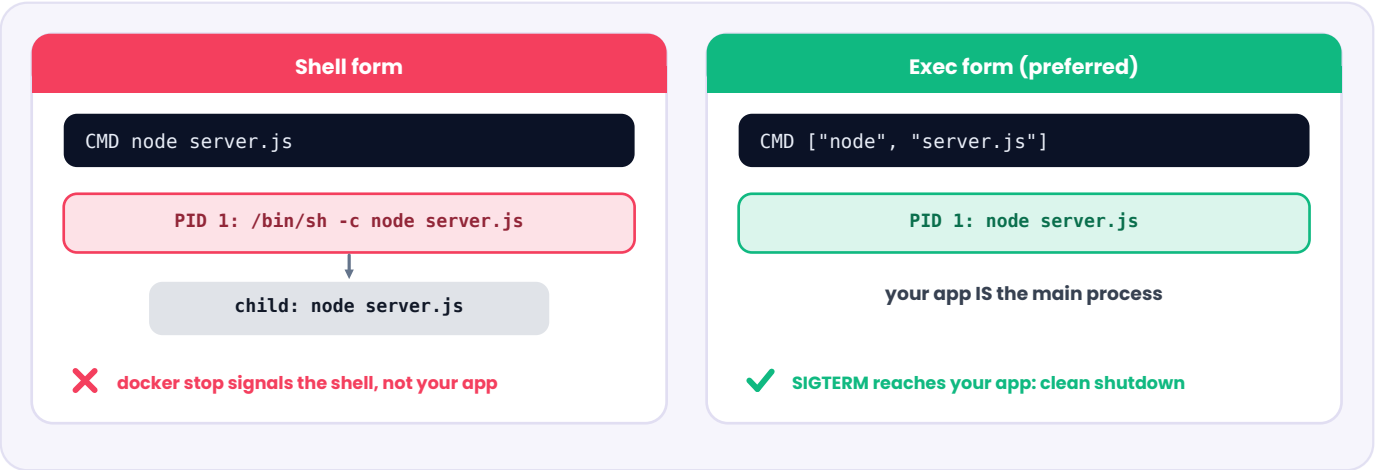


Fig 26. Exec form (a JSON list) makes your app PID 1, so it receives stop signals directly.

CMD vs ENTRYPOINT

Both say what to run. **CMD** is a default you can replace. **ENTRYPOINT** is fixed, and what you type is added to it.

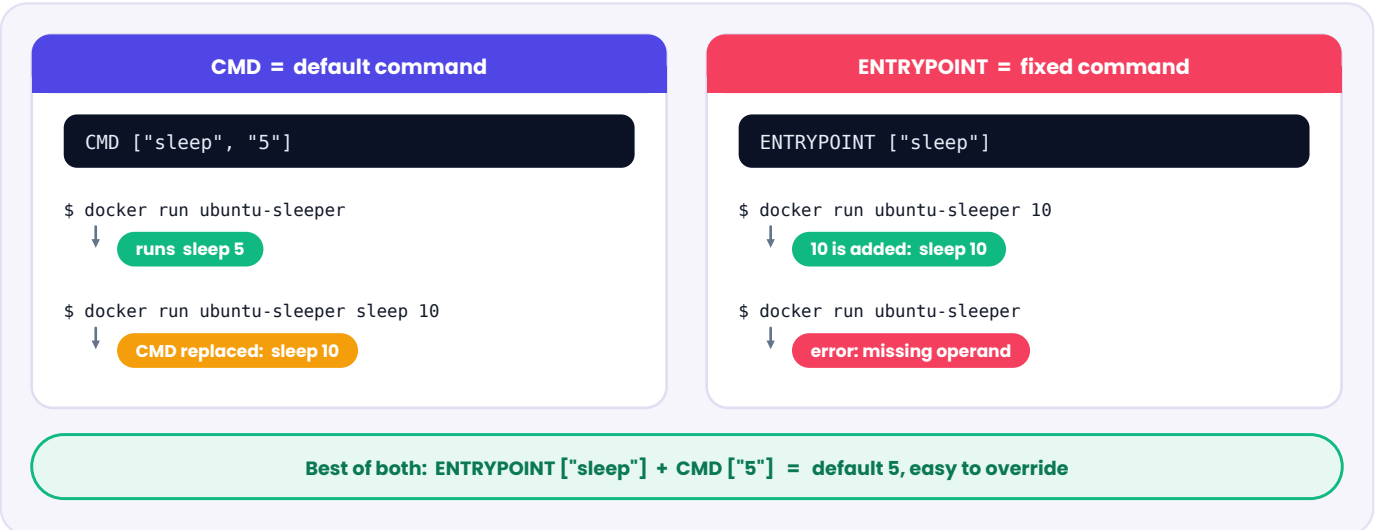


Fig 27. Use ENTRYPOINT for the program and CMD for its default arguments.

```Dockerfile
FROM ubuntu
ENTRYPOINT ["sleep"]
CMD ["5"]
```

```terminal
docker run ubuntu-sleeper          # sleeps 5 seconds (the default)
docker run ubuntu-sleeper 20       # sleeps 20 seconds
docker run --entrypoint sh -it ubuntu-sleeper    # replace the ENTRYPOINT
```

ARG and ENV in a Dockerfile

```Dockerfile
ARG NODE_VERSION=20                  # before FROM: choose the base image
FROM node:${NODE_VERSION}-alpine
ARG APP_ENV=production                # after FROM: usable inside this stage
ENV APP_ENV=$APP_ENV                  # copy it into the runtime environment
```

```terminal
docker build --build-arg NODE_VERSION=22 -t myapp .      # override an ARG at build time
```

HEALTHCHECK

Without a health check Docker only knows the process is *running*. With one, it also knows the app is *working*.

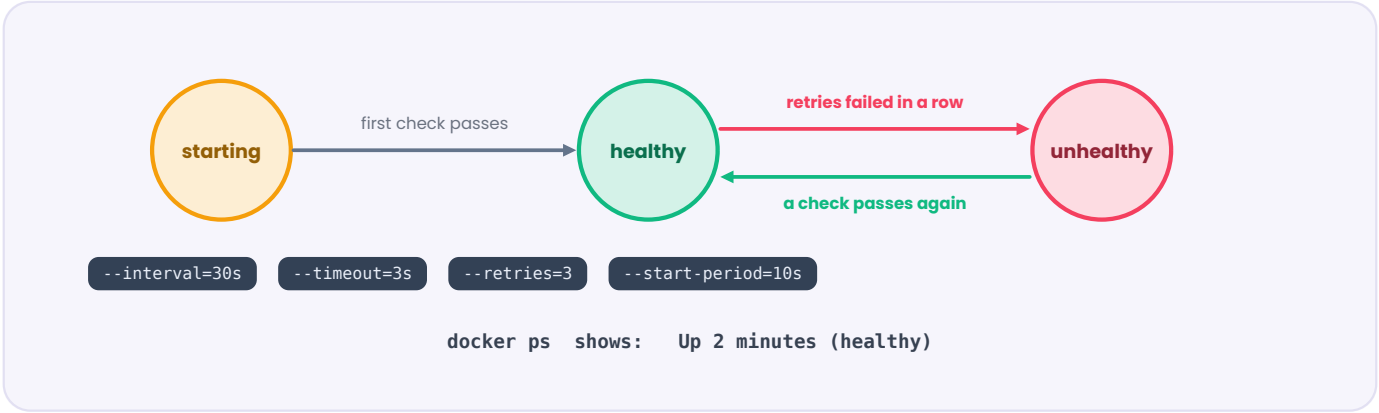


Fig 28. After enough failed checks in a row the container is marked unhealthy.

```Dockerfile
HEALTHCHECK --interval=30s --timeout=3s --retries=3 --start-period=10s \
  CMD curl -f http://localhost:3000/health || exit 1
```

.dockerignore patterns

Pattern	Meaning
`node_modules`	Ignore this folder
`*.log`	Ignore every .log file in the root
`**/*.tmp`	Ignore .tmp files in any folder
`!keep.log`	Exception: do include this one
`.git`	Never send your Git history
`.env`	Never send local secrets

Dockerfile habits worth keeping

- Pin versions: `node:20.11-alpine`, not `node:latest`.
- Copy dependency files first, install, then copy the rest (better cache).
- Chain related commands with `&&` and clean up in the same `RUN`.
- Use the exec (JSON) form for CMD and ENTRYPOINT.
- Run as a non-root `USER`. Never put secrets in `ENV` or `RUN`.
- One main process per container.

13 Building and Optimizing Images

Faster builds, smaller images

Building is one command. Making it **fast and small** takes a few habits.

```
terminal
docker build -t myapp:1.0 .        # -t = name:tag    . = the build context (a folder)
docker run -d -p 3000:3000 myapp:1.0
```

Build option	What it does
`-t name:tag`	Name and tag the result (repeat for more tags)
`-f Dockerfile.dev`	Use a different Dockerfile
`--build-arg KEY=value`	Set an ARG
`--target build`	Stop at a named stage
`--no-cache`	Ignore the cache and rebuild everything
`--pull`	Always fetch a fresh base image
`--platform linux/arm64`	Build for another CPU type
`--progress=plain`	Show full build output (great for debugging)
`--secret id=x,src=file`	Give the build a secret without storing it

The build context

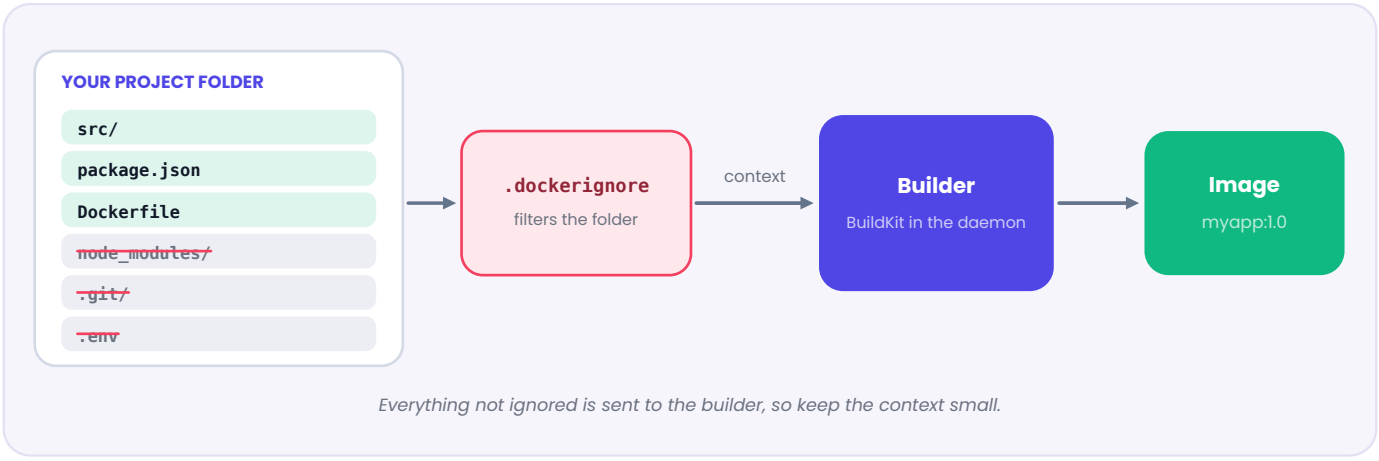


Fig 29. The folder you point at is sent to the builder. A .dockerignore keeps the junk out.

1. Order matters (the layer cache)

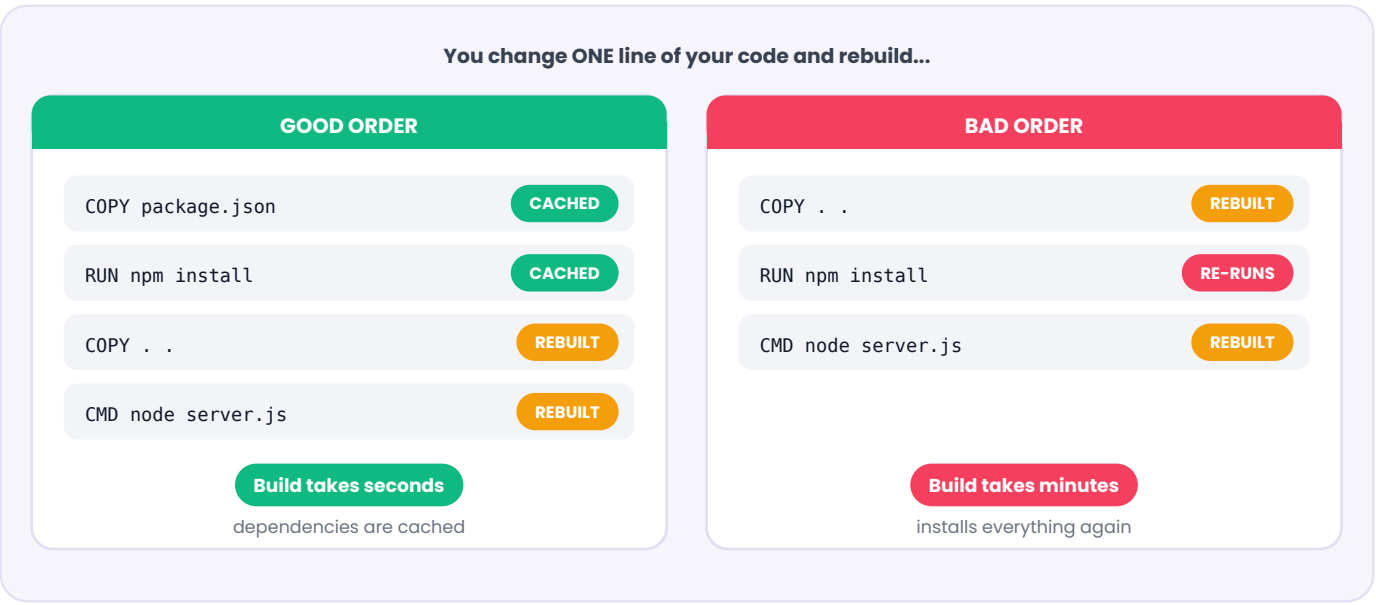


Fig 30. Docker reuses cached layers until something changes. Everything after the change is rebuilt.

TIP

Put what changes **rarely** (dependencies) first and what changes **often** (your code) last.

2. Fewer, cleaner layers

```
Dockerfile
# 3 layers, and the cache stays inside the image
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# 1 layer, clean
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

3. Multi-stage builds

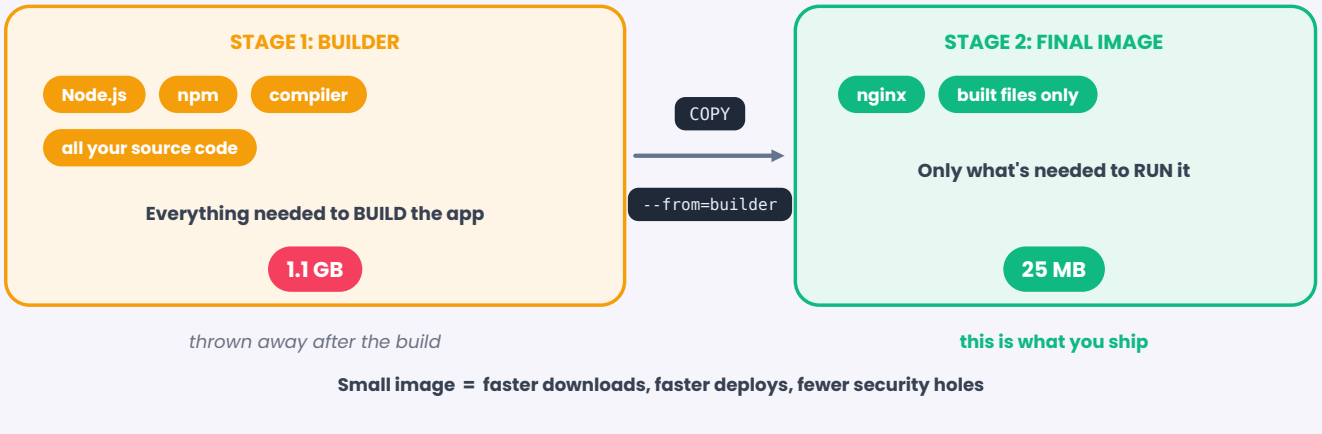


Fig 31. Build with all the tools in stage 1. Ship only the result from stage 2.

```Dockerfile
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm install && npm run build

FROM nginx:alpine AS final
COPY --from=build /app/dist /usr/share/nginx/html
```

```terminal
docker build --target build -t myapp:build .     # build only the first stage (handy for testing)
```

4. Choose a small base image

Base image	About how big	Good for
`scratch`	0 MB	Single static binaries (Go, Rust)
`alpine`	~8 MB	Small general-purpose images
`distroless`	~2 to 20 MB	Apps without a shell or package manager
`debian:slim / ubuntu`	~75 to 80 MB	When you need glibc and common tools
`node:20-alpine`	~135 MB	Node apps
`node:20 (full)`	~1.1 GB	Rarely needed, includes build tools

NOTE

Sizes are rough and change with each release. Check with `docker images`. Alpine uses musl instead of glibc, so a few programs behave differently.

5. BuildKit and buildx

BuildKit is Docker's modern builder (the default). It builds stages in parallel, caches better and adds power-ups. **buildx** is the CLI front end for it.

```Dockerfile
# syntax=docker/dockerfile:1
RUN --mount=type=cache,target=/root/.npm npm ci          # keep the npm cache between builds
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm ci   # secret never lands in a layer
```

```terminal
docker build --secret id=npmrc,src=$HOME/.npmrc -t myapp .
docker buildx ls                                          # list builders
```

Build for several CPU types

Apple Silicon and many cloud servers use **ARM**, while most PCs use **Intel/AMD**. One command can build both.

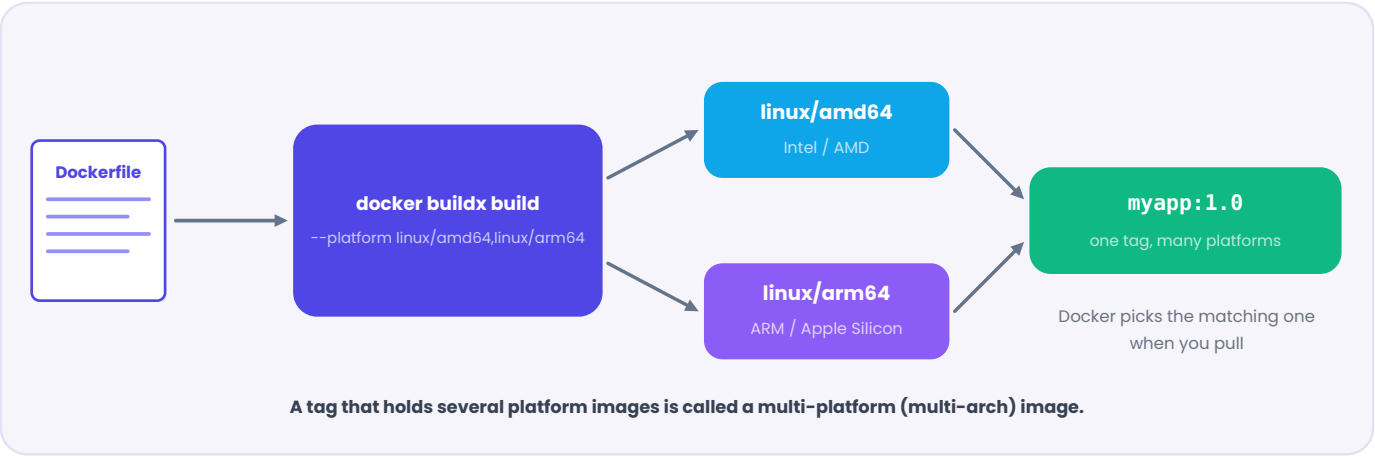


Fig 32. buildx can publish one tag that works on every platform.

```terminal
docker buildx create --use                  # a builder that supports multi-platform
docker buildx build --platform linux/amd64,linux/arm64 -t user/myapp:1.0 --push .
```

TIP

"exec format error" usually means the image was built for a different CPU type than the machine running it.

14 Docker Init

Generate Docker files automatically

`docker init` looks at your project and **writes the Docker files for you**. It is the fastest way to start.

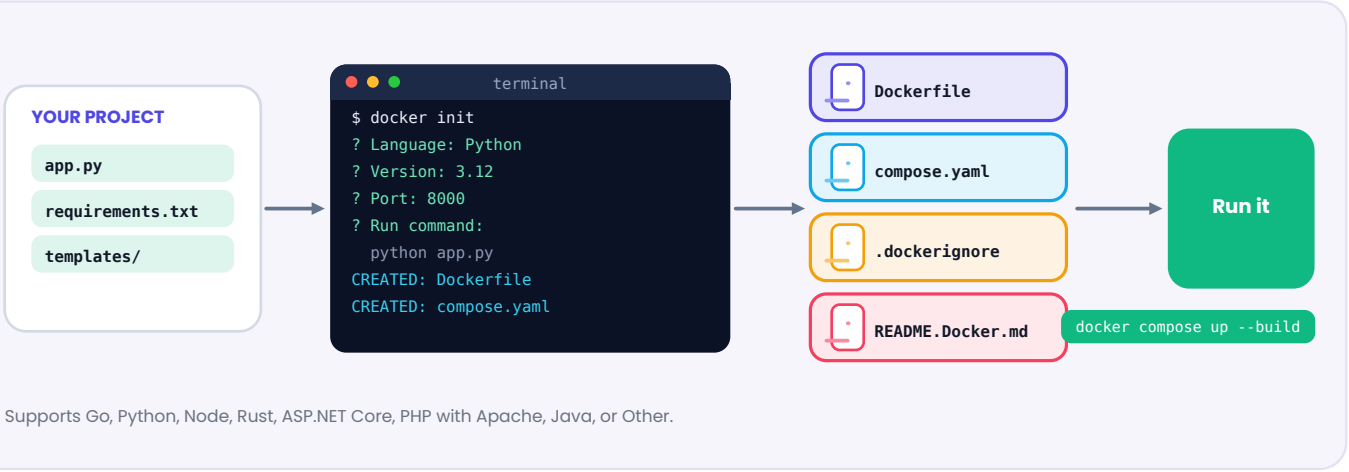


Fig 33. Answer a few questions and docker init creates a working set of Docker files.

THINK OF IT LIKE

Scaffolding or a starter kit. Like `npm init` for packages, but for containers. It uses built-in templates and best practices (it is not guessing with AI), and it asks before overwriting anything.

```
terminal
cd my-project
docker init                    # answer the questions
docker compose up --build      # build the image and run it
```

What it creates

File	What is inside
`Dockerfile`	A tuned Dockerfile for your language: multi-stage, cached dependencies, non-root user
`compose.yaml`	A service for your app with ports and build settings, plus commented examples (for example a database)
`.dockerignore`	Sensible ignores such as .git, node_modules and local environment files
`README.Docker.md`	How to build, run and deploy this project

The questions it asks

Question	Example answer
Which platform or language?	Node, Python, Go, Rust, ASP.NET Core, PHP with Apache, Java, or Other
Which version?	Python 3.12 (it suggests one that matches your project)
Which port does the app use?	8000
How do you start the app?	python app.py (or the framework's command)

Tips

- Run it in the **root of your project**, where your dependency files are.
- **Read the generated files** and adjust them. They are a good start, not the final answer.
- Running `docker init` again asks before it overwrites existing files.
- It ships with Docker Desktop. If `docker init` is not found, update Docker Desktop (or your Docker CLI tools) to a recent version.
- Use `docker init --help` to see what your version supports.

TRY IT

Take any small project, run `docker init`, then `docker compose up --build`. Open the port in your browser and read each generated file to learn how it works.

15 Port Mapping

Letting the outside world in

A container has its **own private network**. To reach an app inside it, you **publish a port** with `-p HOST:CONTAINER`.

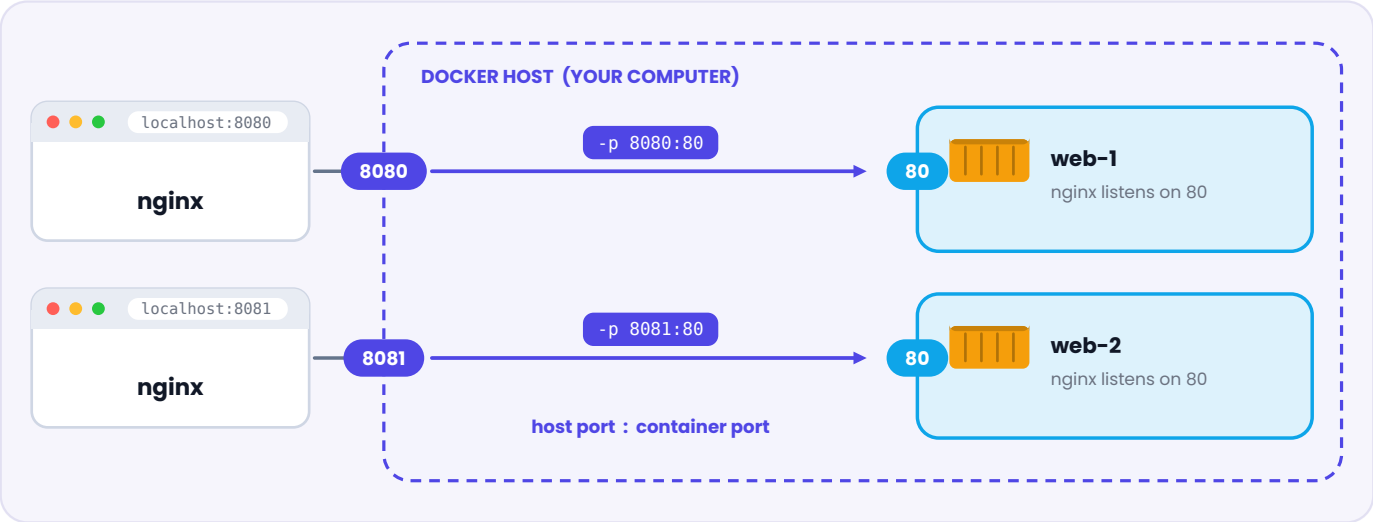


Fig 34. Port 8080 on your computer is forwarded to port 80 inside web-1. Port 8081 goes to web-2.

THINK OF IT LIKE

A company phone system. Callers dial the building's extension (the **host** port) and reception forwards them to the right room (the **container** port).

Flag	Meaning
`-p 8080:80`	Host port 8080 forwards to container port 80
`-p 127.0.0.1:8080:80`	Reachable only from this computer
`-p 5353:53/udp`	Publish a UDP port (TCP is the default)
`-p 8000-8010:8000-8010`	Publish a range of ports
`-p 80`	Container port 80 on a random host port
`-P`	Publish every EXPOSEd port on random host ports

EXPOSE or -p?

	EXPOSE (Dockerfile)	-p (docker run)
What it does	Documents which port the app uses	Actually opens the port to the outside
Reachable from your browser?	No, on its own	Yes
Between containers on one network	Not needed, ports are already reachable	Not needed

```
terminal
docker run -d --name web1 -p 8080:80 nginx
docker run -d --name web2 -p 8081:80 nginx     # same app, different host port
docker port web1                                # show the mappings
curl localhost:8080                             # test it
```

WATCH OUT

Many containers can listen on port 80 **inside**, but each needs its **own host port** outside. If you see "port is already allocated", something is already using that host port.

WATCH OUT

On Linux, Docker publishes ports by editing the firewall (iptables) itself, so a published port can **bypass rules you set in ufw**. Bind to `127.0.0.1` when a port should stay private.

16 Volumes and Data

Keep your data after the container is gone

A container's files **vanish with it**. A **volume** stores data outside the container so it survives.

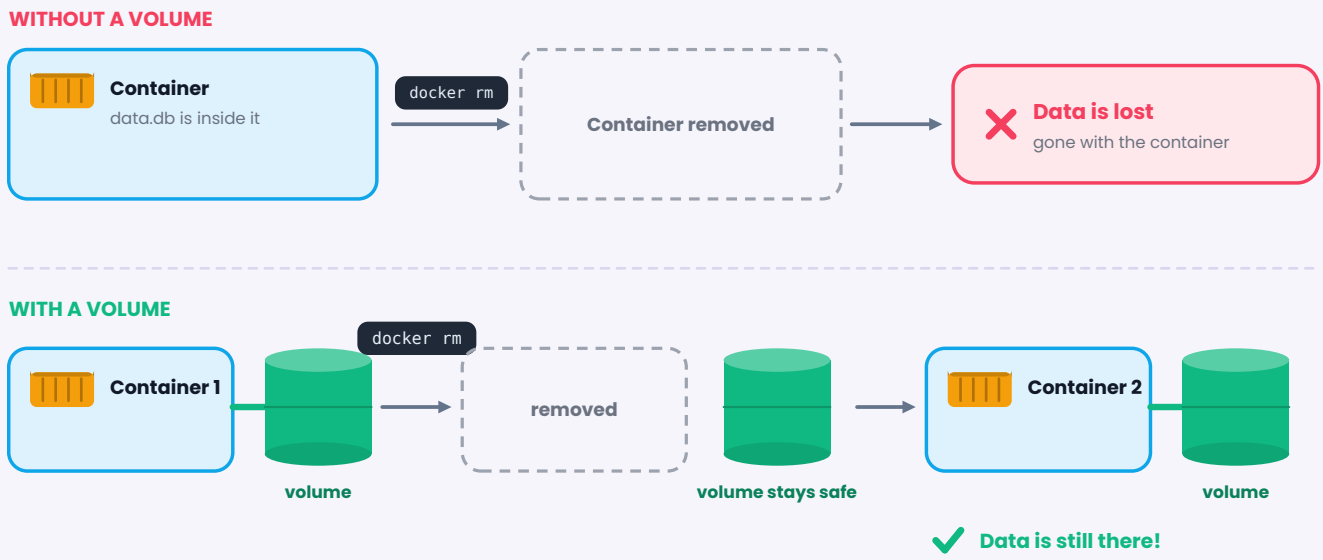


Fig 35. Without a volume, removing the container deletes the data. With one, a new container picks it up.

Three ways to store data

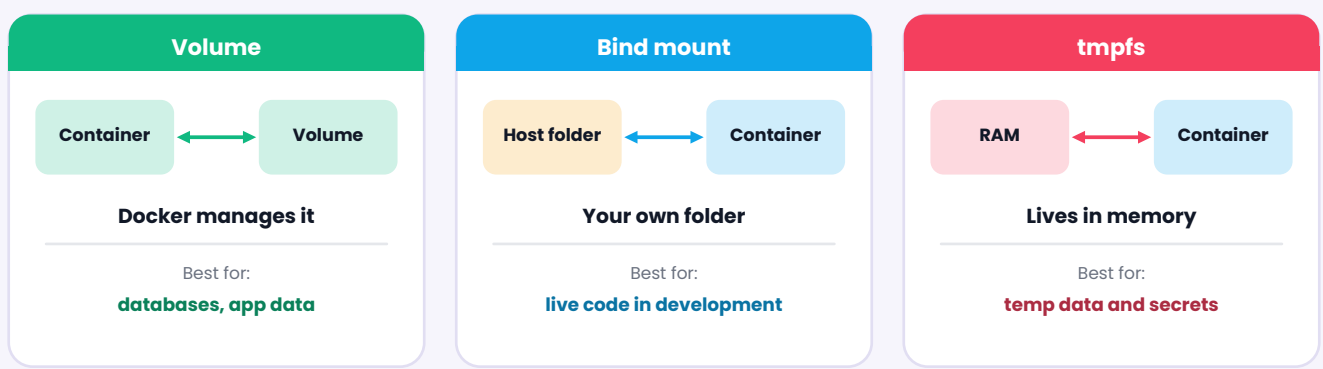


Fig 36. Volumes for app data, bind mounts for your own folders, tmpfs for temporary data.

Type	Created with	Lives	Typical use
Named volume	`-v pgdata:/path`	Managed by Docker	Databases, app data
Anonymous volume	`-v /path`	Managed by Docker, random name	Throwaway data (easy to lose track of)
Bind mount	`-v $(pwd):/app`	A folder you choose	Live code editing
tmpfs	`--tmpfs /tmp`	RAM only	Secrets, scratch files

-v or --mount?

	-v	--mount
Style	Short: source:target:options	Explicit: type=,source=,target=
Missing host folder (bind)	Creates it for you	Gives an error (safer)
Best for	Quick commands	Scripts and clarity

```
terminal
docker run -d -v pgdata:/var/lib/postgresql/data postgres              # short form
docker run -d --mount type=volume,source=pgdata,target=/var/lib/postgresql/data postgres
docker run -d -v pgdata:/data:ro nginx                  # :ro = read-only inside the container
docker run -it --volumes-from db alpine sh              # reuse another container's volumes
```

Manage volumes

`docker volume create v`
make a volume

`docker volume ls`
list volumes

`docker volume inspect v`
where is it?

`docker volume rm v`
delete (must be unused)

`docker volume prune`
delete all unused

`docker system df -v`
volume sizes

Back up and restore a volume

```
terminal
# backup: tar the volume into the current folder
docker run --rm -v pgdata:/data -v "$(pwd)":/backup alpine tar czf /backup/pgdata.tar.gz -C /data .

# restore into a (new) volume
docker run --rm -v pgdata:/data -v "$(pwd)":/backup alpine tar xzf /backup/pgdata.tar.gz -C /data
```

WATCH OUT

Bind mounts use **your** file permissions. If the app inside runs as another user you may see "permission denied". On SELinux systems (Fedora, RHEL) add `:z` to the mount, like `-v ./src:/app:z`.

REAL EXAMPLE

Running a database in a container? **Always** mount a volume, or every removed container wipes your data.

17 Networking Basics

How containers talk to each other

Docker creates **virtual networks**. Containers on the same **custom network** find each other **by name**, with no IP addresses to remember.

Networks that exist from day one

```
terminal
docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
a1b2c3d4e5f6   bridge    bridge    local      # the default network for new containers
b2c3d4e5f6a1   host      host      local
c3d4e5f6a1b2   none      null      local
```

Default bridge vs your own bridge

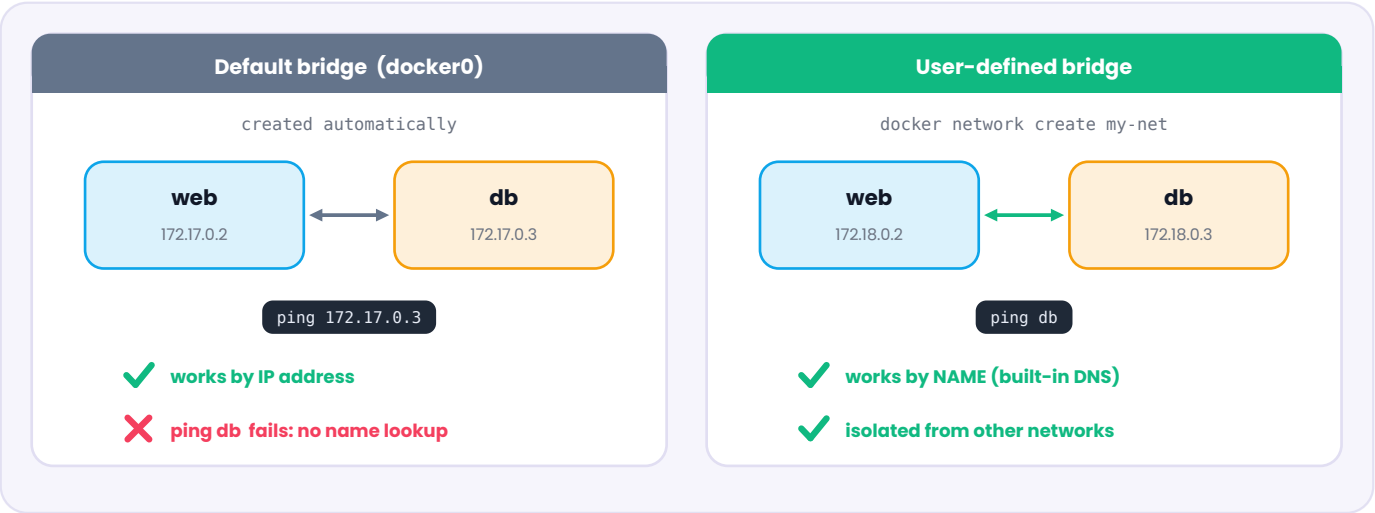


Fig 37. Always create your own network. It adds name lookup and better isolation.

	Default bridge	User-defined bridge
Find containers by name	No, IP addresses only	Yes, built-in DNS
Isolation	All containers share it	Only members can talk
Attach or detach live	Must recreate the container	docker network connect / disconnect
Good for	Quick experiments	Real applications

A small application network

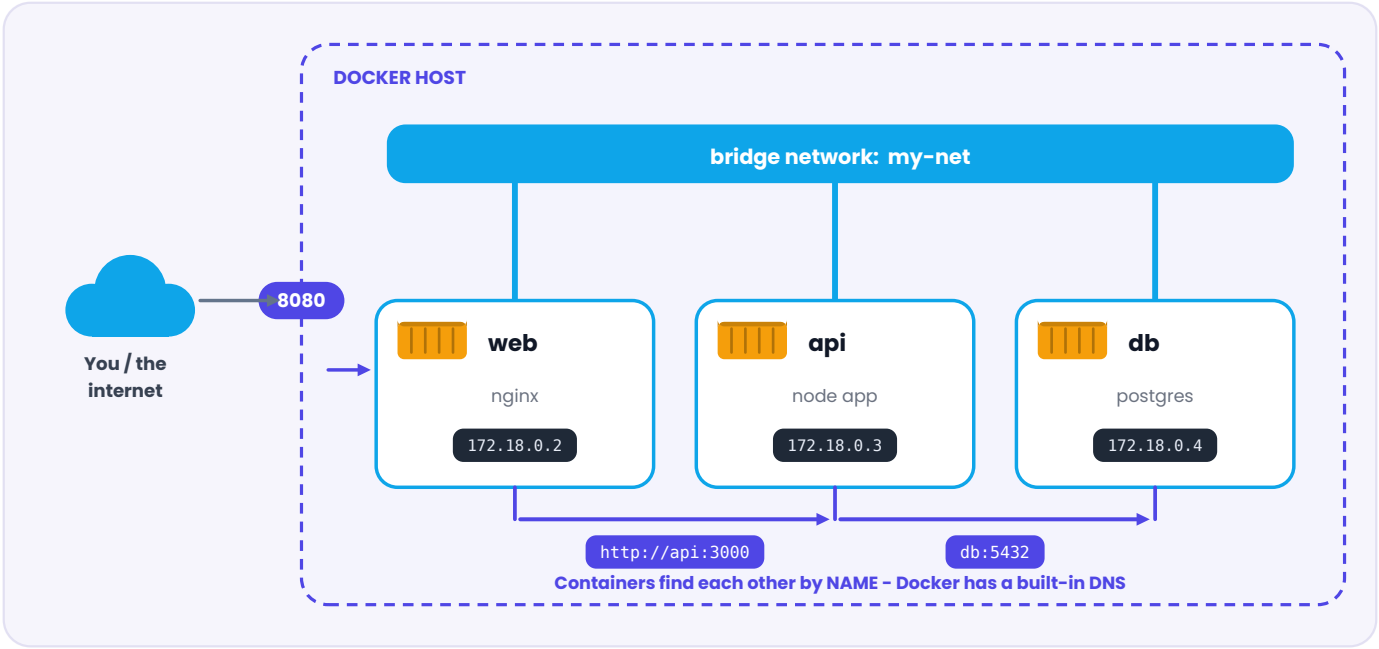


Fig 38. web, api and db share the my-net network. The outside world only reaches web, through port 8080.

```terminal
docker network create my-net
docker run -d --name db  --network my-net postgres
docker run -d --name api --network my-net my-api      # api reaches the database at db:5432
docker run -d --name web --network my-net -p 8080:80 my-web
```

Useful options

Option	What it does
`--network-alias db`	An extra name other containers can use for this one
`--hostname app1`	The hostname the container sees for itself
`--ip 172.20.0.10`	A fixed IP (needs a custom network with a subnet)
`--dns 1.1.1.1`	A custom DNS server for the container
`--add-host x:10.0.0.5`	Add a line to the container's /etc/hosts

```terminal
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web     # a container's IP
docker exec web ping db                                                              # test name lookup
```

TIP

Reaching a service on your **host** from a container? Docker Desktop provides `host.docker.internal`. On Linux add `--add-host=host.docker.internal:host-gateway`. Every custom network has a built-in DNS server, which is why `db` works as a hostname.

18 Network Drivers and Legacy

Every driver, plus what is outdated

Docker ships several **network drivers**. Most apps only need **bridge**, but you should know every option, including the old ones you will still meet in tutorials.

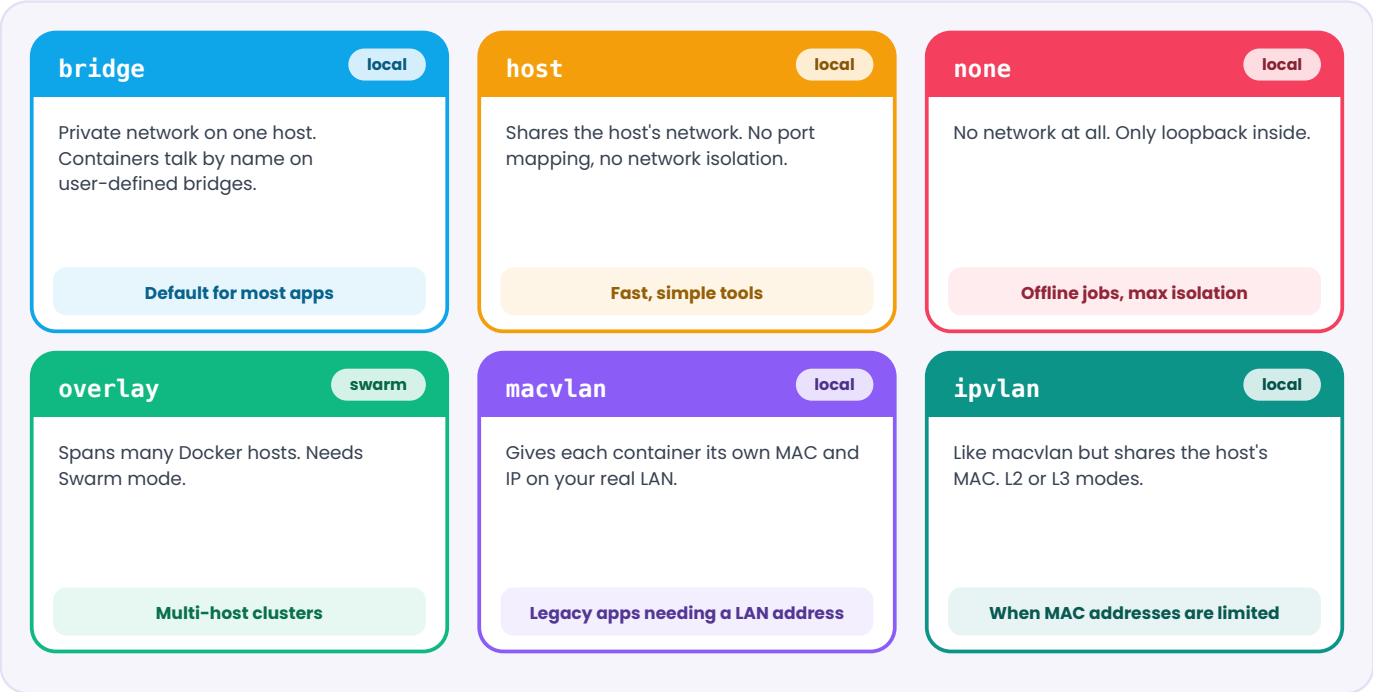


Fig 39. Rule of thumb: user-defined bridge for almost everything, overlay for Swarm, the rest only for a specific reason.

Driver	Scope	Isolation	Example
`bridge`	One host	Good	`docker network create my-net`
`host`	One host	None (shares host stack)	`docker run --network host nginx`
`none`	One host	Complete	`docker run --network none alpine`
`overlay`	Swarm cluster	Good, can be encrypted	`docker network create -d overlay app-net`
`macvlan`	One host	Container sits on your LAN	`docker network create -d macvlan ...`
`ipvlan`	One host	Like macvlan, shared MAC	`docker network create -d ipvlan ...`

macvlan and ipvlan: containers on your real network

```terminal
docker network create -d macvlan --subnet=192.168.1.0/24 --gateway=192.168.1.1 \
  -o parent=eth0 lan-net
docker run -d --network lan-net --ip 192.168.1.50 nginx       # reachable at 192.168.1.50 on your LAN

docker network create -d ipvlan -o ipvlan_mode=l2 --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 -o parent=eth0 lan-net2
```

WATCH OUT

With macvlan the host itself usually **cannot** talk to its own macvlan containers, and many Wi-Fi adapters and cloud networks do not allow it. It works best on wired LANs.

Host and overlay in practice

```
terminal
docker run -d --network host nginx            # nginx listens on the host's port 80 directly, -p is ignored
docker swarm init                             # overlay networks need Swarm mode
docker network create -d overlay --attachable app-net
```

NOTE

Host networking is a **Linux** feature. On Docker Desktop (Mac and Windows) it refers to the Docker VM and is opt-in in newer versions, so use `-p` there.

Sharing one network with another container

A container can **borrow another container's network**. It is still current, and it is the same idea Kubernetes uses for pods.

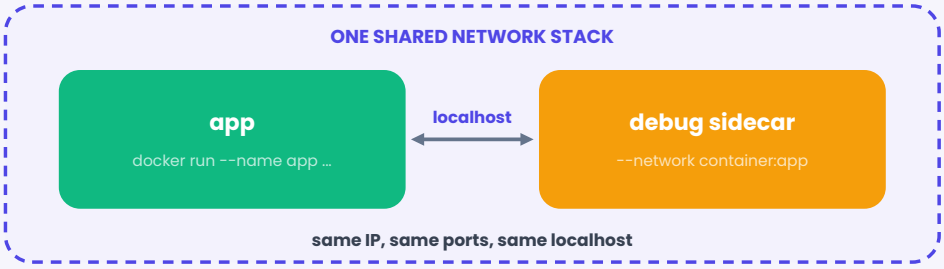


Fig 40. Both containers share one IP address and one localhost.

```
terminal
docker run -d --name app my-app
docker run -it --network container:app nicolaka/netshoot     # debug tools inside app's network
```

Networks Docker creates for itself

Name	Purpose
`bridge (docker0)`	The default network. Linux shows it as the docker0 interface
`ingress`	Swarm's routing mesh, so a published port works on every node
`docker_gwbridge`	Lets Swarm overlay networks reach the outside world

Outdated or rarely used, and what replaced them

Older guides still show these. They work in some cases, but there is a better way today.

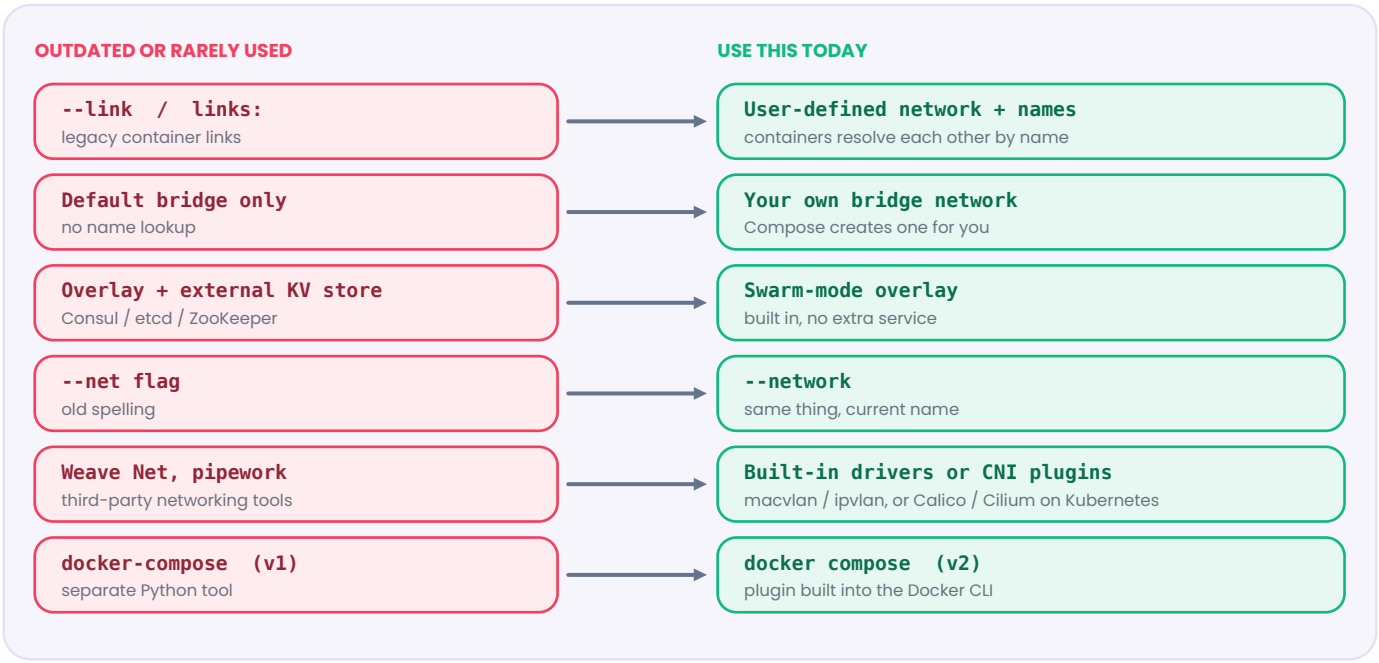


Fig 41. When you see something on the left in a tutorial, use the right-hand version instead.

```
terminal
# OLD: legacy link (deprecated)
docker run -d --name db postgres
docker run -d --name api --link db:database my-api

# NEW: shared user-defined network, names just work
docker network create my-net
docker run -d --name db  --network my-net postgres
docker run -d --name api --network my-net my-api     # reach it as  db
```

Network commands

docker network create -d driver --subnet --gateway	**docker network ls** list networks	**docker network inspect n** subnet, members, settings
docker network connect n c attach a running container	**docker network disconnect n c** detach it	**docker network prune** remove unused networks

create option	What it does
`--subnet 172.20.0.0/16`	Pick the address range
`--gateway 172.20.0.1`	Pick the gateway address
`--internal`	No access to the outside world (private backend)
`--attachable`	Let standalone containers join an overlay network
`--ipv6`	Enable IPv6 on this network

19 Environment Variables and Config

One image, many settings

Do not hard-code settings like colors, URLs or passwords. **Pass them in** when the container starts, and one image works everywhere.

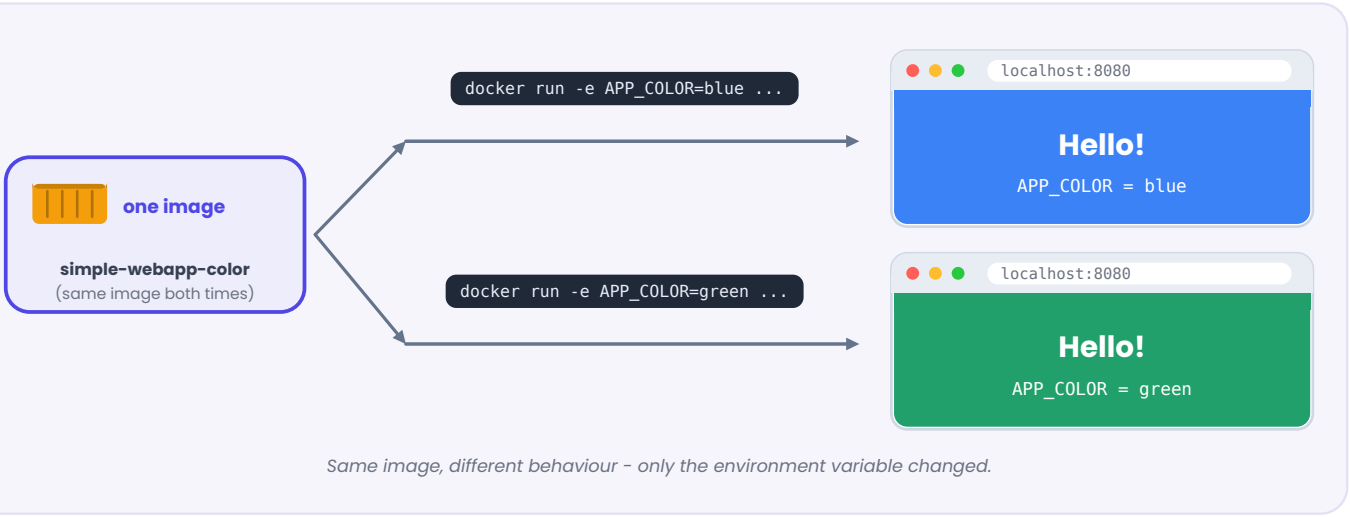


Fig 42. The same image shows a different page depending on the environment variable you pass.

```terminal
docker run -e APP_COLOR=blue myapp          # one variable
docker run -e HOME myapp                    # no value: copies HOME from your shell
docker run --env-file .env myapp           # many variables from a file
docker exec web env                        # see what the container received
```

The env file format

```.env
# .env  (one KEY=value per line)
APP_COLOR=blue
DB_HOST=db
DB_PORT=5432
```

> **NOTE**
> In a plain `--env-file` quotes are **not** removed and there is no variable substitution. Keep it simple: `KEY=value`. Add `.env` to `.gitignore` and `.dockerignore`.

Who wins when a variable is set twice?

Highest priority first	Example
1. docker run -e	`-e APP_COLOR=red`
2. docker run --env-file	`--env-file .env`
3. ENV in the Dockerfile	`ENV APP_COLOR=blue`

ARG vs ENV

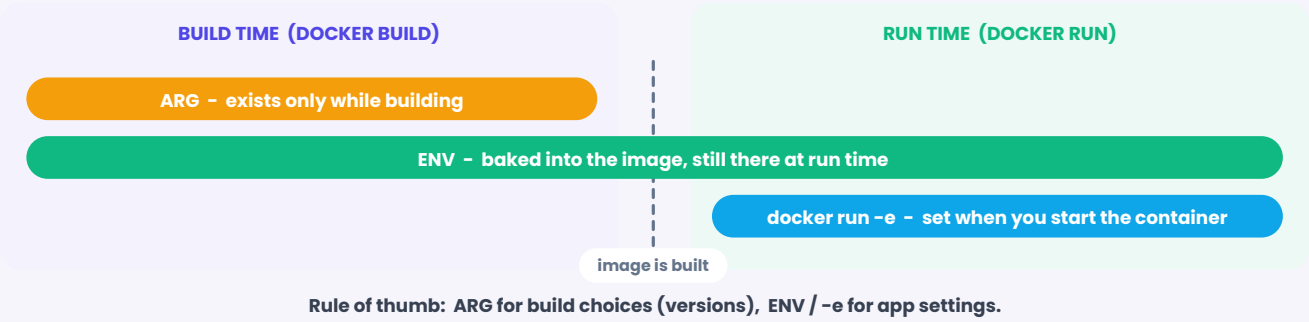


Fig 43. ARG exists only during the build. ENV stays in the image and is visible at run time.

Where should each kind of setting go?

Kind of setting	Best place
Version to build with (Node 20)	ARG + --build-arg
Normal app settings (port, mode)	ENV default + -e or env file
Different per environment (dev, prod)	Env file or Compose environment
Passwords and API keys	Docker secrets or injected at run time, never inside the image

WATCH OUT

Never put passwords or API keys in a Dockerfile (`ENV` or `RUN`). They stay in the image layers forever, and `docker inspect` shows environment variables to anyone who can run it.

20 YAML in Two Minutes

The format Compose files use

Compose files are written in **YAML**, a plain-text format for settings. It looks like an outline: **indentation shows structure**. You only need a handful of rules.

```
compose.yaml

# a comment

name: demo

services:

   web:

      image: nginx:1.27

      ports:

         - "8080:80"

      environment:

         MODE: production

      command: ["nginx", "-g", "daemon off;"]
```

#
a comment, ignored by YAML

key: value
one setting. The space after : is required

indent
2 spaces = one level deeper. Never tabs

- item
a list item: dash, then a space

[a, b]
the same list written on one line

"..."
quote values with a colon, like "8080:80"

text
unquoted yes, no, on, off can turn into booleans

Fig 44. The seven YAML ideas you need for Compose files.

Three kinds of values

Kind	Looks like	Meaning
Map	`key: value`	A named setting. A map can hold more maps indented below it
List	`- item`	An ordered list. Each item starts with a dash and a space
Scalar	`nginx:1.27, 8080, true`	A single value: text, number or true/false

The same idea written two ways

```
compose.yaml
environment:                # map style
  MODE: production
  DEBUG: "false"

environment:                # list style (the same meaning)
  - MODE=production
  - DEBUG=false
```

compose.yaml

```
command: ["nginx", "-g", "daemon off;"]       # a list on one line
command:                                      # the same list, one item per line
  - nginx
  - -g
  - daemon off;
```

Mistakes everyone makes once

Mistake	What happens	Fix
Tabs for indentation	Parse error	Use spaces only (2 per level)
image:nginx	Not a key: value pair	Space after the colon: image: nginx
ports: - 22:22	YAML may read it as a number	Quote it: "22:22"
no, yes, on, off	Turn into true/false	Quote them when you mean text
Uneven indentation	Setting lands in the wrong place	Keep siblings aligned
Text with : or #	Cut off or misread	Wrap in quotes

TIP

Not sure your file is valid? Run `docker compose config`. It prints the final merged file, or tells you exactly which line is wrong.

NOTE

Comments start with #. Use them freely. Compose ignores them, and your future self will thank you.

21 Docker Compose Basics

Run a whole app with one command

Real apps use several containers: a web app, a database, a cache. **Compose** describes them all in **one YAML file** and starts everything with one command.

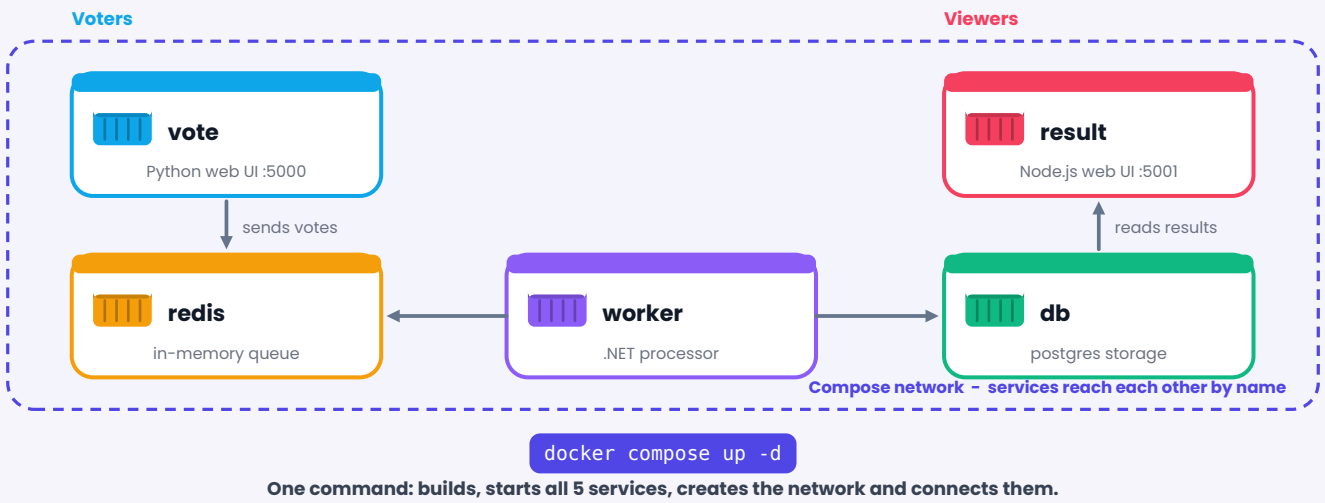


Fig 45. A voting app with five services. Compose creates one network and they find each other by name.

Anatomy of a Compose file

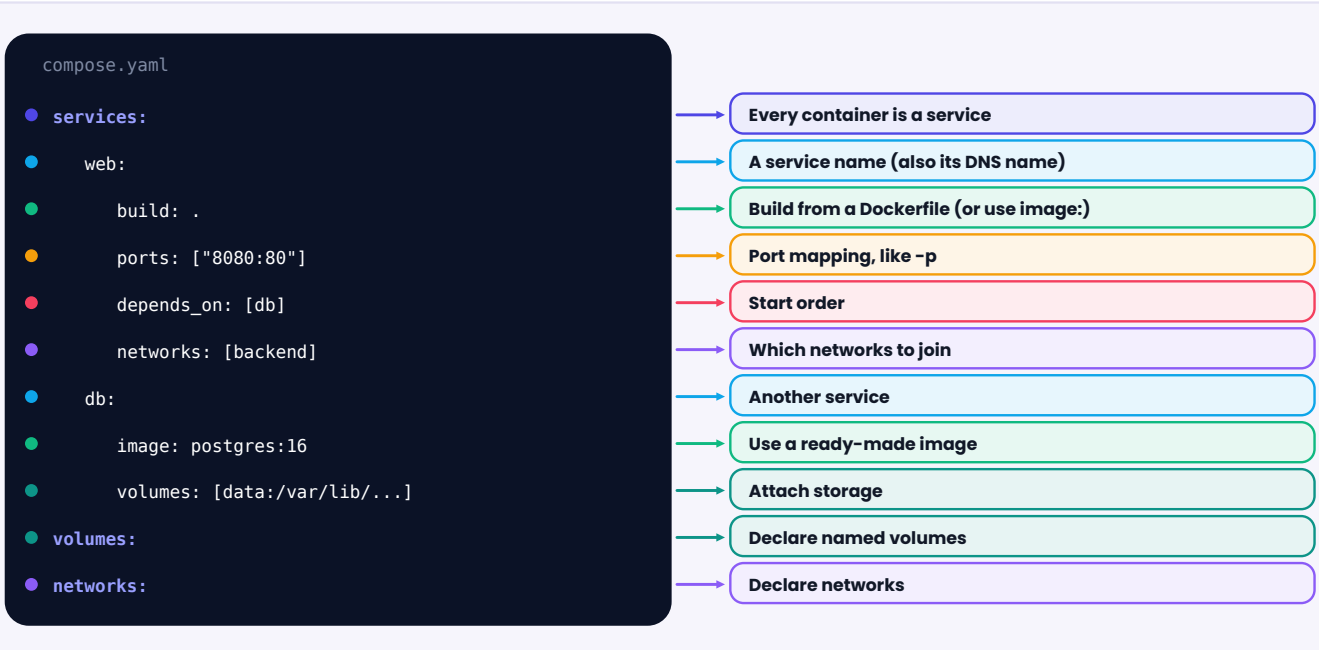


Fig 46. The five top-level blocks: services, volumes, networks (plus secrets and configs when needed).

What Compose does for you

Step	What happens
1	Reads compose.yaml (also found as compose.yml or docker-compose.yml)
2	Builds the images that use build: and pulls the ones that use image:
3	Creates a network named *project*_default and any volumes you declared
4	Starts the containers in dependency order, named *project*-*service*-1

compose.yaml

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

The core commands

`docker compose up -d`
start everything

`docker compose down`
stop and remove

`docker compose ps`
list the services

`docker compose logs -f`
follow all logs

`docker compose build`
rebuild the images

`docker compose exec web sh`
shell into a service

NOTE

Use `docker compose` (with a space). The old `docker-compose` tool (version 1) is retired, and the `version:` line at the top of a file is obsolete. The project name defaults to the folder name (change it with `-p` or `name:`).

REAL EXAMPLE

New teammate? `git clone` then `docker compose up`. The whole stack runs, with nothing else to install.

22 Compose File Reference

Every important key, explained

Every important key of a service, explained. First a complete example, then the details.

compose.yaml

```yaml
services:                          # every container you want is a service
  web:                             # the service name (also its hostname on the network)
    build:                         # build an image from source instead of pulling one
      context: .                   # folder sent to the builder
      dockerfile: Dockerfile       # which Dockerfile to use
      args:                        # build arguments (ARG)
        NODE_VERSION: "20"
    image: myapp:1.0               # name given to the built image
    container_name: web            # fixed name (skip it if you want to scale)
    command: ["node", "server.js"]  # replaces the image's CMD
    working_dir: /app              # starting folder inside the container
    user: "1000:1000"              # run as this user
    restart: unless-stopped        # restart policy
    ports:                         # publish ports, HOST:CONTAINER
      - "8080:3000"
    environment:                   # environment variables
      DB_HOST: db
    env_file: .env                 # or load them from a file
    volumes:                       # storage
      - ./src:/app/src             # bind mount: your folder into the container
      - uploads:/app/uploads       # named volume
    networks: [frontend, backend]  # which networks it joins
    depends_on: [db]               # start order
    healthcheck:                   # how Docker tests that it works
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
```

Where does the container come from?

Key	What it does
`image`	Image to pull or the name for a built image
`build`	Build from a Dockerfile. Sub-keys: context, dockerfile, args, target, platforms
`pull_policy`	always, missing, never or build: when to pull
`platform`	Force a platform such as linux/amd64

How does it run?

Key	What it does
`command`	Replace the image's default CMD
`entrypoint`	Replace the image's ENTRYPOINT
`container_name`	Fixed container name (blocks scaling)
`working_dir / user`	Start folder and user inside the container
`restart`	no, on-failure, always or unless-stopped
`init`	Run a tiny init as PID 1 (reaps zombie processes)
`tty / stdin_open`	Same as -t and -i, for interactive containers
`stop_signal / stop_grace_period`	Signal and wait time used when stopping

How is it connected?

Key	What it does
`ports`	Publish ports to the host: "8080:80"
`expose`	Document an internal port (does not publish it)
`networks`	Which networks to join, optionally with aliases
`hostname / dns`	Hostname and DNS servers
`extra_hosts`	Extra /etc/hosts lines, like --add-host

Data and configuration

Key	What it does
`volumes`	Named volumes, bind mounts, :ro read-only
`environment`	Variables as a map or a list
`env_file`	Load variables from one or more files
`secrets / configs`	Mount sensitive values or config files (under /run/secrets)
`tmpfs`	Memory-only folders
`labels`	Metadata, also used by other tools

Limits and security

Key	What it does
`deploy.resources.limits`	cpus and memory limits for the service
`mem_limit / cpus`	Short forms of the same limits
`read_only`	Read-only root filesystem
`cap_drop / cap_add`	Remove or add Linux capabilities
`security_opt`	For example no-new-privileges:true
`logging`	Log driver and options for this service
`privileged`	Almost all host powers. Avoid it

Compose key vs docker run flag

Compose key	docker run equivalent
`image`	`the image argument`
`container_name`	`--name`
`ports`	`-p`
`environment / env_file`	`-e / --env-file`
`volumes`	`-v`
`networks`	`--network`
`restart`	`--restart`
`user / working_dir`	`-u / -w`
`command / entrypoint`	`trailing command / --entrypoint`
`healthcheck`	`--health-cmd and friends`
`deploy.resources.limits`	`--memory / --cpus`
`read_only`	`--read-only`

TIP

If you know `docker run`, you already know Compose. Every key is just a flag written down in a file.

23 Compose: depends_on and Health

Start things in the right order

A database container can be **running** long before it is **ready**. **depends_on** plus a **healthcheck** make Compose wait for the right moment.

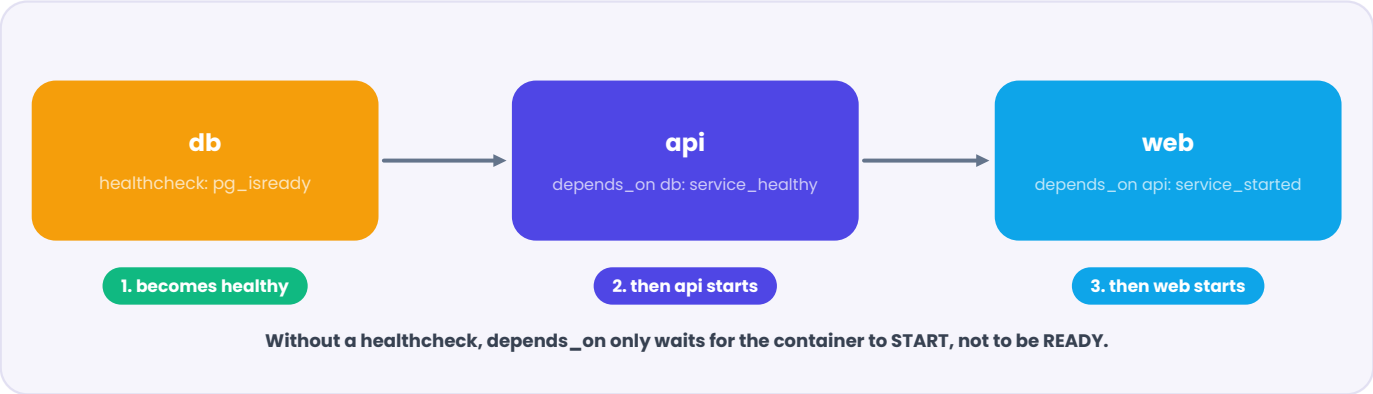


Fig 47. api waits until db reports healthy. web waits until api has started.

depends_on: short and long syntax

```yaml
compose.yaml
services:
  web:
    depends_on: [api]                  # short: wait until api has STARTED

  api:
    depends_on:                        # long: choose a condition
      db:
        condition: service_healthy     # wait until db's healthcheck passes
      migrate:
        condition: service_completed_successfully   # wait for a one-off job to finish
        restart: true                  # restart api if db is restarted
        required: true                 # false = start even if this one is missing
```

Condition	Compose waits until
`service_started`	The dependency's container has started (the default)
`service_healthy`	Its healthcheck reports healthy
`service_completed_successfully`	It ran and exited with code 0 (for migrations and setup jobs)

healthcheck: every line explained

compose.yaml

```
db:
  image: postgres:16
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]   # the command Docker runs; exit 0 = healthy
    interval: 10s       # how often to run it
    timeout: 5s         # how long to wait for one run to answer
    retries: 5          # failed runs in a row before 'unhealthy'
    start_period: 20s   # grace time while the app boots (failures do not count)
```

Key	Meaning
`test`	The check. ["CMD", ...] runs a program, ["CMD-SHELL", "..."] runs a shell line, ["NONE"] turns it off
`interval`	Time between checks (for example 30s or 1m30s)
`timeout`	A check that takes longer than this counts as failed
`retries`	Consecutive failures needed to become unhealthy
`start_period`	Boot-up grace time
`disable`	true switches off a healthcheck from the image

Healthcheck commands for popular images

Service	test
PostgreSQL	`["CMD-SHELL", "pg_isready -U postgres"]`
MySQL / MariaDB	`["CMD", "mysqladmin", "ping", "-h", "localhost"]`
Redis	`["CMD", "redis-cli", "ping"]`
A web app	`["CMD-SHELL", "curl -f http://localhost:3000/health || exit 1"]`

NOTE

The command runs **inside** the container, so the tool must exist there. A slim image may have no `curl`. Use `wget`, the app's own CLI, or install curl.

WATCH OUT

Even with `service_healthy`, write apps that **retry** their connections. Databases can restart at any time.

See the result

terminal

```
docker compose up -d
docker compose ps                 # STATUS shows: Up 30 seconds (healthy)
docker inspect db --format '{{.State.Health.Status}}'     # healthy, unhealthy or starting
```

24 Compose: Volumes, Networks, Vars

Storage, isolation and settings

Volumes, networks and variables get their own blocks in a Compose file. Here is how each one works.

Volumes

```yaml
compose.yaml
services:
  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data    # named volume (managed by Docker)
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro   # bind mount, read-only
  web:
    volumes:
      - type: bind                         # long syntax: the same thing, spelled out
        source: ./src
        target: /app/src
        read_only: true
volumes:
  pgdata:                                  # declare named volumes here
  shared:
    external: true                         # created outside Compose, not deleted by it
```

Networks

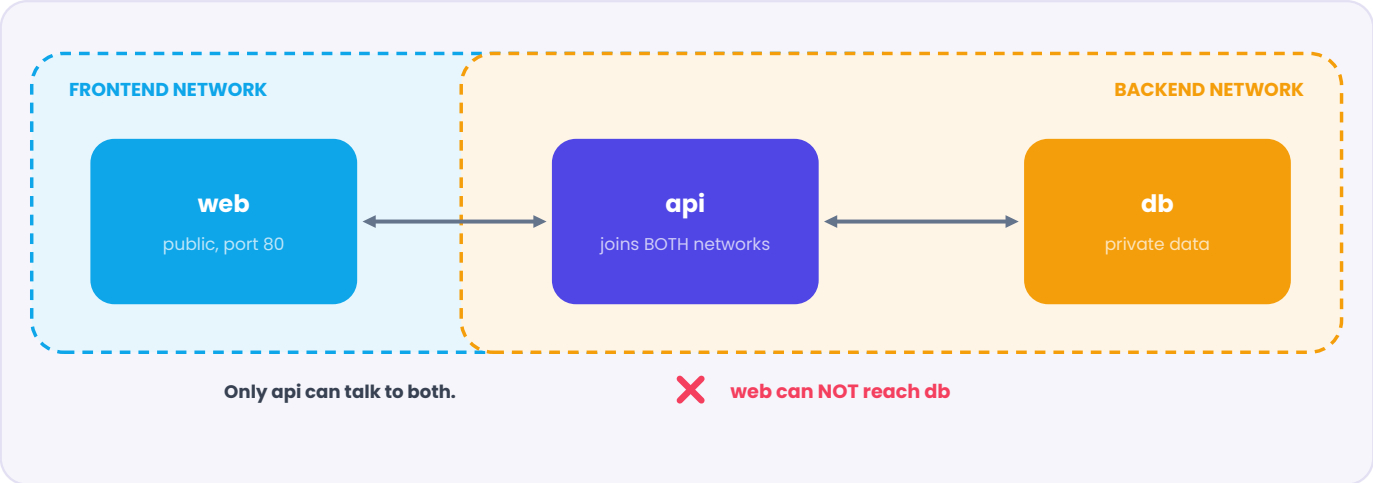


Fig 48. web and db live on different networks, so they cannot reach each other. api bridges them.

```yaml
compose.yaml
services:
  web: { networks: [frontend] }
  api: { networks: [frontend, backend] }
  db:  { networks: [backend] }
networks:
  frontend:
  backend:
    internal: true            # no internet access from this network
```

> **NOTE**
> Without a `networks:` block Compose creates one default network and every service joins it. Services always find each other by **service name**.

Variables

Where	What it is for
`environment:`	Variables passed into the container
`env_file:`	Load container variables from a file
`.env file (next to compose.yaml)`	Values for ${VAR} placeholders **in the Compose file itself**
`Your shell`	Also feeds ${VAR}, and wins over the .env file

compose.yaml

```
# .env
TAG=1.0
DB_PASSWORD=s3cret

# compose.yaml
services:
  web:
    image: myapp:${TAG:-latest}          # use $TAG, or 'latest' if it is not set
    environment:
      DB_PASSWORD: ${DB_PASSWORD:?set DB_PASSWORD}     # stop with an error if it is missing
```

Syntax	Meaning
`${VAR}`	Use the value of VAR
`${VAR:-default}`	Use default when VAR is unset or empty
`${VAR:?message}`	Stop with message when VAR is unset or empty
`$$`	A literal dollar sign

Secrets and configs

compose.yaml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password    # secrets appear as files in /run/secrets (not in docker inspect)
    secrets: [db_password]
secrets:
  db_password:
    file: ./db_password.txt                               # the secret lives in a local file
```

25 Compose Workflow and Commands

Profiles, overrides and watch

Compose commands, and the features that make larger projects easier.

Command	What it does
`up -d`	Create and start everything in the background
`up --build`	Rebuild images before starting
`up --force-recreate`	Recreate containers even if nothing changed
`up -d --scale web=3`	Run 3 copies of the web service
`down`	Stop and remove containers and networks (keeps volumes)
`down -v`	Also delete named volumes (deletes data!)
`start / stop / restart`	Control containers without removing them
`ps -a`	List services and their status
`logs -f --tail 50 web`	Follow the last 50 lines of one service
`exec web sh`	Shell inside a running service
`run --rm web npm test`	One-off container for a single command
`build --no-cache`	Rebuild images from scratch
`pull / push`	Download or upload the images
`config`	Print the final merged file (and check it)
`top / images / port`	Processes, images used, port mappings
`cp web:/f ./`	Copy files between a service and your folder
`ls`	List all Compose projects on this machine

Profiles: services that only start when asked

compose.yaml

```
services:
  web:
    image: myapp
  adminer:                          # a database admin tool for debugging
    image: adminer
    profiles: ["debug"]             # skipped by default
```

terminal

```
docker compose up -d                      # starts web only
docker compose --profile debug up -d      # starts web and adminer
```

Several files: base plus overrides

```terminal
docker compose up -d                                          # reads compose.yaml AND compose.override.yaml
docker compose -f compose.yaml -f compose.prod.yaml up -d   # choose the files yourself
docker compose -f compose.yaml -f compose.prod.yaml config   # preview the merged result
```

```compose.yaml
# compose.yaml can also pull in other files
include:
  - ./db/compose.yaml
```

Develop with live updates: compose watch

```compose.yaml
services:
  web:
    build: .
    develop:
      watch:
        - action: sync              # copy changed files into the running container
          path: ./src
          target: /app/src
        - action: rebuild           # rebuild the image when this file changes
          path: package.json
```

```terminal
docker compose watch                 # or: docker compose up --watch
```

TIP

Scaling: do not publish a fixed host port or set `container_name` on a service you scale. Copies cannot share them.

KEY IDEA

`down` removes containers but keeps volumes. `down -v` also removes volumes. `stop` only stops.

A complete example: web, api, database and cache

compose.yaml

```yaml
name: shop
services:
  web:
    build: ./web
    ports: ["8080:80"]
    depends_on:
      api: { condition: service_started }
    networks: [frontend]
  api:
    build: ./api
    environment:
      DB_HOST: db
      REDIS_HOST: cache
    depends_on:
      db: { condition: service_healthy }
      cache: { condition: service_started }
    networks: [frontend, backend]
    restart: unless-stopped
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5
    networks: [backend]
  cache:
    image: redis:7-alpine
    networks: [backend]
volumes:
  pgdata:
networks:
  frontend:
  backend:
secrets:
  db_password:
    file: ./db_password.txt
```

26 Docker Hub

The default public registry

Docker Hub (hub.docker.com) is the default public registry. When you run `docker pull nginx`, this is where Docker looks.

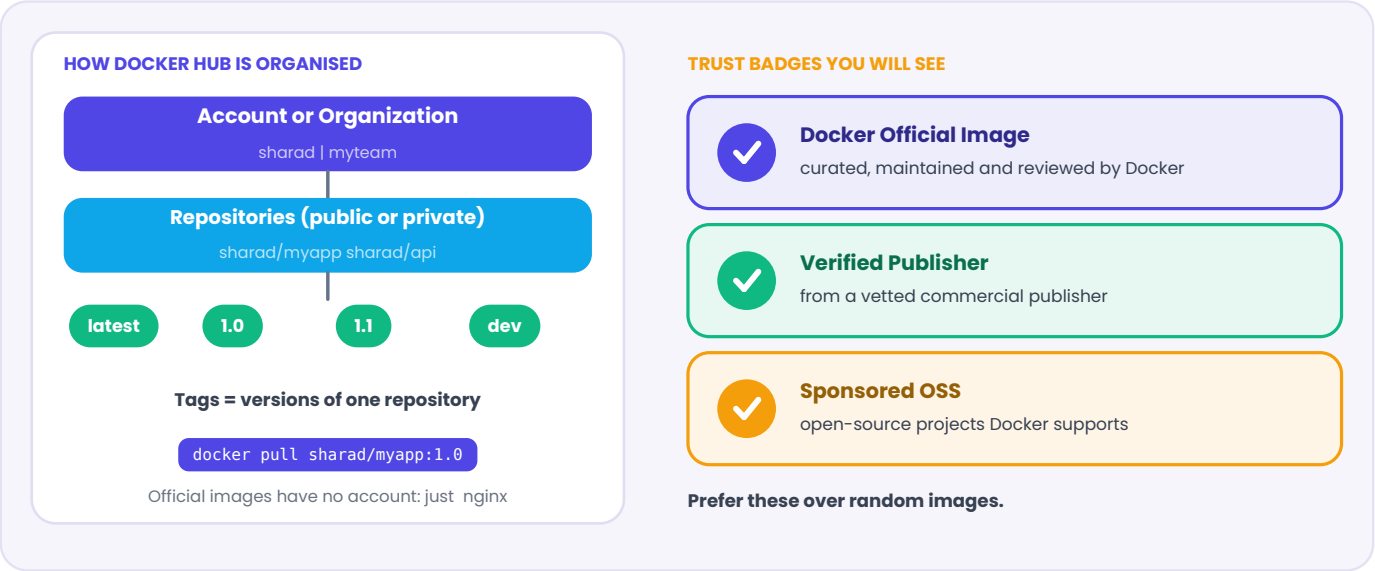


Fig 49. Hub holds accounts and organizations. Each owns repositories, and each repository holds tags.

Publish your first image

```
terminal
docker login                                # sign in (browser or username + token)
docker tag myapp:1.0 sharad/myapp:1.0       # the name must start with YOUR Docker Hub username
docker push sharad/myapp:1.0                # upload
docker pull sharad/myapp:1.0                # download anywhere
docker logout
```

Step	Where	What to do
1	hub.docker.com	Create a free account
2	Repositories	Create a repository and choose public or private
3	Your terminal	docker login, tag the image with username/repo, then push
4	Repository page	Add a README so others know how to use it

Log in with a token, not your password

A **personal access token** can be limited to read-only or read-write and revoked at any time. Create one in *Account settings, Personal access tokens*.

```
terminal
echo $DOCKER_TOKEN | docker login -u sharad --password-stdin     # token comes from stdin, not shell history
```

Names and tags on Docker Hub

You type	It really means
`nginx`	`docker.io/library/nginx:latest (an Official Image)`
`bitnami/redis:7`	`docker.io/bitnami/redis:7 (a publisher's image)`
`sharad/myapp:1.0`	`docker.io/sharad/myapp:1.0 (your own image)`

```terminal
docker tag myapp:1.0 sharad/myapp:1.0       # the exact version
docker tag myapp:1.0 sharad/myapp:1             # a moving 'major version' tag
docker tag myapp:1.0 sharad/myapp:latest        # what people get with no tag
docker push --all-tags sharad/myapp               # push them all
```

Search from the terminal

```terminal
docker search nginx --filter is-official=true --limit 5
```

Pull limits

Docker Hub limits how many images one client can pull in a window of time. **Logging in raises the limit.**

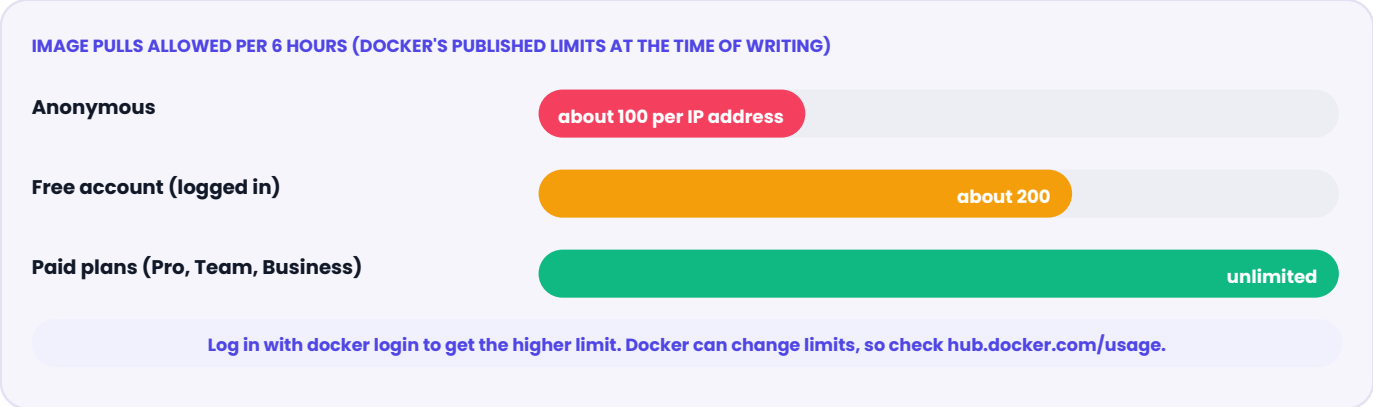


Fig 50. Limits are Docker's published numbers at the time of writing and can change.

```terminal
# Many pulls from one network? Use a mirror or your own cache.
docker run -d -p 5000:5000 --name mirror -e REGISTRY_PROXY_REMOTEURL=https://registry-1.docker.io registry:2
# then in /etc/docker/daemon.json:   { "registry-mirrors": ["http://localhost:5000"] }
```

More Hub features

Feature	What it gives you
Organizations and teams	Share repositories with a group and give each team read, write or admin rights
Repository visibility	Public for everyone, private for invited people. How many private repos you get depends on your plan
Webhooks	Hub calls a URL of yours whenever a new image is pushed
Tags tab	See every tag, its size, supported platforms and push time
Docker Scout results	Vulnerability information for your images (see the Docker Scout section)
Official and Verified badges	Signals that an image is maintained by Docker or a vetted publisher

TIP

Prefer **Docker Official Images** and **Verified Publishers**. Anyone can upload an image to Hub, so check who published it, how often it is updated and how many people use it.

27 GitHub Container Registry

Images next to your code

GitHub Container Registry (**ghcr.io**) stores Docker images next to your code on GitHub. It works with the same `docker login`, `push` and `pull` commands.

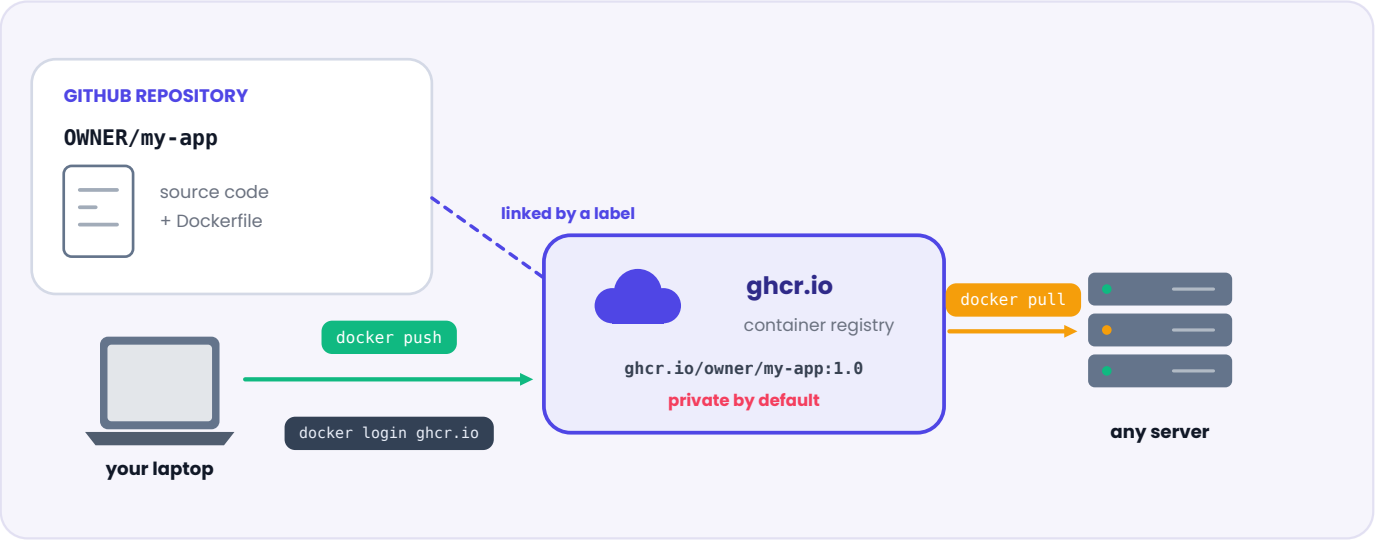


Fig 51. An image lives at ghcr.io/OWNER/NAME. A label in the Dockerfile links it to its repository.

Step by step

Step	What to do
1	On GitHub create a **personal access token (classic)**: Settings, Developer settings, Personal access tokens
2	Tick **write:packages** to push and pull (add read:packages to only pull, delete:packages to delete)
3	Log in to ghcr.io with your GitHub username and the token
4	Tag with **ghcr.io/OWNER/IMAGE:TAG** (all lowercase), then push

```terminal
echo $GITHUB_PAT | docker login ghcr.io -u YOUR_USERNAME --password-stdin
docker tag myapp:1.0 ghcr.io/your-username/myapp:1.0        # owner and image name must be lowercase
docker push ghcr.io/your-username/myapp:1.0
docker pull ghcr.io/your-username/myapp:1.0
docker logout ghcr.io
```

Visibility and linking

Topic	How it works
Default visibility	New packages are **private**
Make it public	Package page, Package settings, Change visibility. Public images can be pulled without logging in
Link to a repository	Add a label in the Dockerfile and rebuild

```Dockerfile
LABEL org.opencontainers.image.source="https://github.com/OWNER/REPO"      # in your Dockerfile
```

GitHub Container Registry or Docker Hub?

	GitHub (ghcr.io)	Docker Hub
Address	ghcr.io	docker.io (the default)
Image name	ghcr.io/owner/image:tag	user/image:tag, or just image for official ones
Log in with	GitHub username + classic token	Docker ID + password or access token
New images are	Private until you change it	Public or private, you choose
Best for	Images that belong with a GitHub project	Sharing publicly, official base images

WATCH OUT

A token is a password. Send it through `--password-stdin`, never type it into the command line, and never commit it to a repository.

NOTE

The token for ghcr.io must be a **classic** personal access token. If a pull says "denied", check the scope and that the package is not private to someone else.

28 Private and Other Registries

Run your own or use another

Docker Hub and GitHub are not your only options. Companies often run **private registries**, and you can run your own in one command.

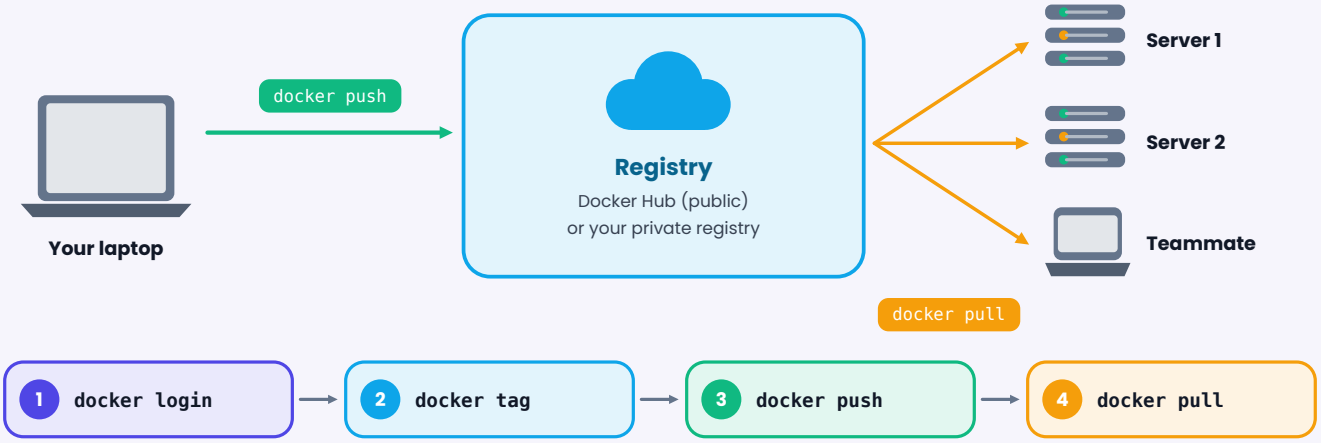


Fig 52. Push from your laptop. Pull on any server or from a teammate's machine.

Registries you may meet

Registry	Address looks like	Good for
Docker Hub	`docker.io/user/app`	Public sharing, official images
GitHub Container Registry	`ghcr.io/owner/app`	Images next to GitHub code
GitLab Container Registry	`registry.gitlab.com/group/app`	Images next to GitLab code
Amazon ECR	`123456.dkr.ecr.region.amazonaws.com/app`	Images on AWS
Google Artifact Registry	`region-docker.pkg.dev/project/repo/app`	Images on Google Cloud
Azure Container Registry	`name.azurecr.io/app`	Images on Azure
Quay.io	`quay.io/org/app`	Public and private hosting
Harbor	`your own address`	Self-hosted with access control and scanning

Run your own registry

```
terminal
docker run -d -p 5000:5000 --name registry -v registry-data:/var/lib/registry registry:2
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
curl localhost:5000/v2/_catalog                # list what is stored
docker pull localhost:5000/myapp:1.0
```

> **NOTE**
> The `-v registry-data` volume keeps your images when the registry container is replaced. Docker expects **HTTPS** for remote registries. `localhost` is the exception.

A registry on another machine without HTTPS (testing only)

```
daemon.json
# /etc/docker/daemon.json
{ "insecure-registries": ["myregistry.local:5000"] }
```

> **WATCH OUT**
> `insecure-registries` turns off encryption checks. Use real certificates for anything that is not a throwaway test.

Logging in to any registry

```
terminal
docker login registry.example.com
docker pull registry.example.com/team/app:1.0
cat ~/.docker/config.json        # where Docker remembers your logins
```

> **TIP**
> Docker can keep logins in your operating system's secure store (the `credsStore` setting in `config.json`) instead of the file itself. Never commit `config.json` to Git.

29 Docker Scout

Find vulnerabilities in images

Docker Scout inspects your image, lists every package inside it and tells you which ones have **known security problems** (CVEs), and what to do about them.

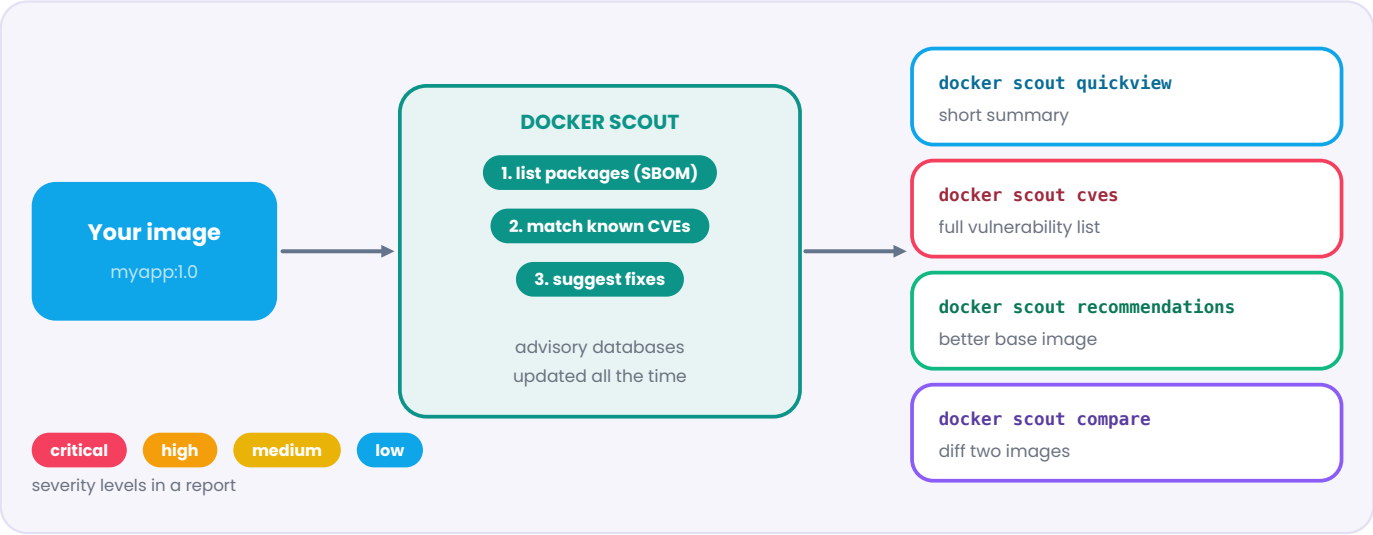


Fig 53. Scout builds a package list (SBOM), matches it against advisory databases and gives you a report.

The commands

`docker scout quickview IMG`
short summary

`docker scout cves IMG`
full CVE list

`docker scout recommendations IMG`
better base image

`docker scout compare IMG --to OLD`
what changed

`docker scout sbom IMG`
list all packages

`docker scout policy IMG`
check against policies

```
terminal
docker scout quickview myapp:1.0                    # start here
docker scout cves myapp:1.0                         # every vulnerability found
docker scout cves --only-severity critical,high myapp:1.0     # just the serious ones
docker scout cves --only-fixed myapp:1.0             # only issues that already have a fix
docker scout recommendations myapp:1.0               # which base image to move to
docker scout compare myapp:2.0 --to myapp:1.0       # did the new build get better?
docker scout cves archive://app.tar                  # scan a saved image file
```

Where Scout can look

Source	Means
`local://myapp:1.0`	An image on this machine (the default)
`registry://nginx:1.27`	An image in a registry, without pulling it
`archive://app.tar`	An image saved with docker save

Reading a report

Term	Meaning
CVE	A public ID for a known vulnerability, like CVE-2024-12345
Severity	Critical, high, medium or low. Fix critical and high first
Affected package	The library inside your image that has the problem
Fixed version	The version that removes the problem. No fixed version means nothing to upgrade to yet
Base image	Many problems come from the image after FROM. Updating it often fixes them all

A simple routine

Step	Do this
1	docker scout quickview to see the big picture
2	docker scout cves --only-fixed to see what you can fix today
3	docker scout recommendations, then update the FROM line and rebuild
4	docker scout compare new --to old to prove it improved

Where else you will see Scout

- **Docker Desktop:** the Images view shows a vulnerability summary for each image.
- **Docker Hub:** repositories with Scout turned on show results on the repository page.
- **Turn it on for a repository:** `docker scout enroll ORG` then `docker scout repo enable --org ORG ORG/REPO`.

NOTE

Scout comes with Docker Desktop and as a Docker CLI plugin. Some features and the number of repositories you can monitor depend on your Docker plan. Run `docker scout --help` to see what your version offers.

TIP

A report is a **starting point**. Not every CVE can actually be exploited in your app, so start with critical and high issues that have a fix.

30 Logging and Monitoring

See what your containers are doing

Containers should write logs to **stdout and stderr**. Docker collects them, and you read them with `docker logs`.

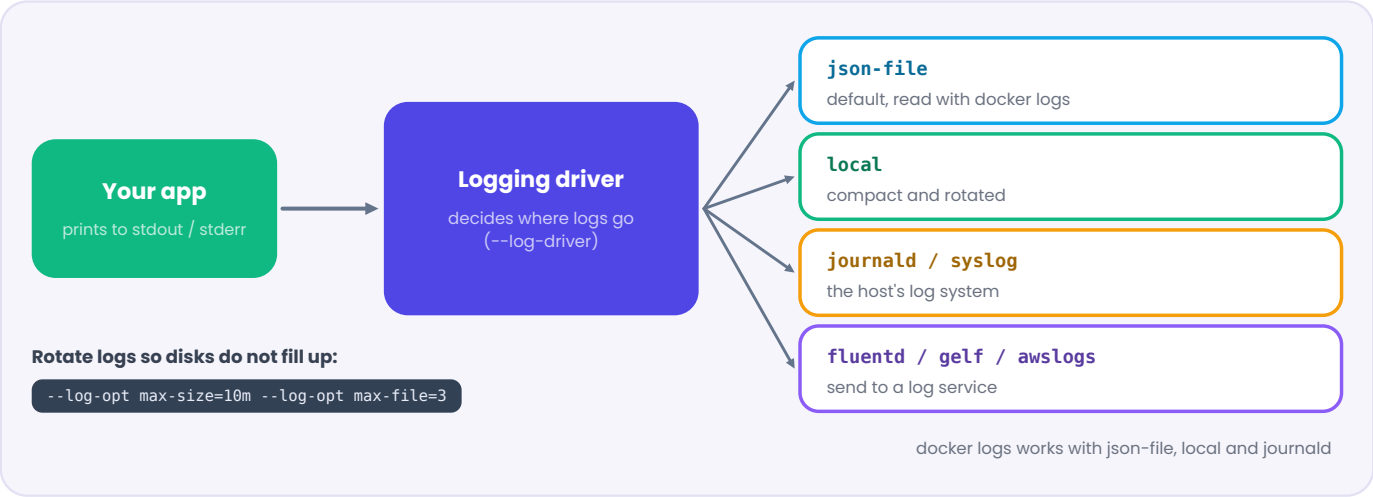


Fig 54. A logging driver decides where a container's output is stored or sent.

Reading logs

```
terminal
docker logs web                  # everything so far
docker logs -f web               # follow live (Ctrl+C to stop watching)
docker logs --tail 50 web        # only the last 50 lines
docker logs --since 10m web      # the last 10 minutes (also: 2h, or a date)
docker logs -t web               # add timestamps
docker compose logs -f web api   # several services at once
```

Logging drivers

Driver	What it does
`json-file`	The default. Stores JSON files on the host, works with docker logs
`local`	Compact, rotated files. A good modern default
`journald`	Sends logs to the systemd journal
`syslog`	Sends logs to a syslog server
`fluentd, gelf, splunk`	Ship logs to a log platform
`awslogs, gcplogs`	Ship logs to cloud logging services
`none`	Discard logs

```
terminal
docker run -d --log-driver json-file --log-opt max-size=10m --log-opt max-file=3 nginx    # per container
# or for every container, in /etc/docker/daemon.json:
{ "log-driver": "json-file", "log-opts": { "max-size": "10m", "max-file": "3" } }
```

WATCH OUT

The default json-file driver **never rotates** unless you set `max-size`. A chatty container can fill the disk. Set rotation in `daemon.json`.

Watching resources and events

```
terminal
docker stats                          # live CPU, memory, network and disk for every container
docker stats --no-stream              # one snapshot, then exit
docker top web                        # the processes inside a container
docker events --filter event=die      # live feed of Docker events (here: containers that stopped)
docker inspect web --format '{{.State.Health.Status}}'     # health status
```

docker stats column	Meaning
CPU %	Share of CPU in use right now
MEM USAGE / LIMIT	Memory used, and the limit set with --memory
NET I/O	Data received and sent over the network
BLOCK I/O	Data read from and written to disk
PIDS	Number of processes and threads

Tools you can add later

Tool	Role
cAdvisor	Collects container CPU, memory and network metrics
Prometheus	Stores those metrics over time
Grafana	Draws dashboards from them
Loki or an ELK stack	Collects and searches logs

TIP

The Docker Desktop dashboard shows logs, CPU and memory for each container with no extra setup.

31 Troubleshooting and Cleanup

Find and fix problems step by step

When something breaks, do not guess. **Walk the checklist** from top left to bottom right.

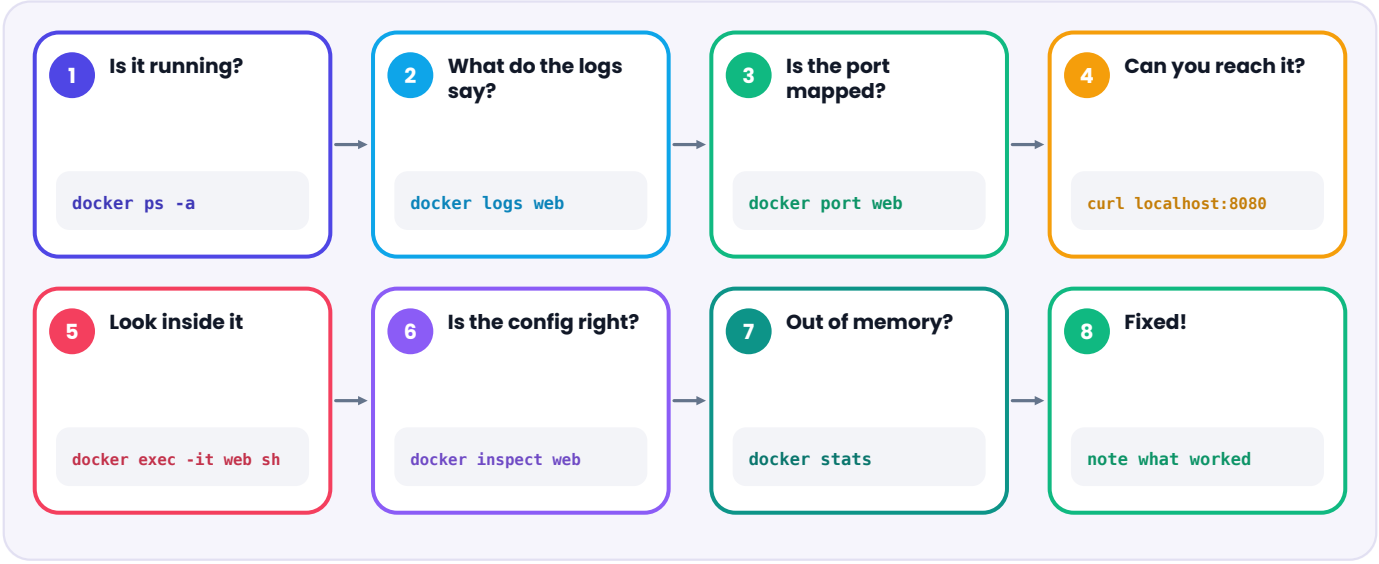


Fig 55. The troubleshooting checklist as a flow.

Common errors and fixes

You see	Likely cause and fix
Cannot connect to the Docker daemon	The daemon is not running. Start Docker Desktop or `sudo systemctl start docker`
permission denied ... docker.sock	Add yourself to the docker group, then log out and in: `sudo usermod -aG docker $USER`
port is already allocated	Another program or container uses that host port. Pick another `-p` or stop the other one
container name is already in use	An old container has that name. `docker rm -f name` or choose a new name
pull access denied / not found	Typo in the image name, or a private image that needs `docker login`
exec format error / no matching manifest	The image is for another CPU type. Use `--platform linux/amd64` or a multi-platform image
Container exits right away	Its main process ended. Check `docker logs`. Shell images need `-it`
executable file not found	Wrong path in CMD or ENTRYPOINT, or the tool is missing from the image
Connection refused between containers	Different networks, wrong name, or the app listens on 127.0.0.1 instead of 0.0.0.0
permission denied on a volume	File owner mismatch. Check `USER` and folder permissions
no space left on device	Run `docker system df`, then prune

Debugging toolkit

```
terminal
docker logs --tail 100 web                      # what did it say?
docker inspect web                              # config, mounts, networks, exit code
docker exec -it web sh                          # look inside a running container
docker run --rm -it --entrypoint sh myapp:1.0   # start the image with a shell instead of the app
docker exec web ping db                         # can it reach the database by name?
docker network inspect my-net                   # who is on the network?
docker events                                   # what is Docker doing right now?
```

TIP

A web app must listen on `0.0.0.0` inside the container. If it listens only on `127.0.0.1`, even a correct `-p` mapping cannot reach it.

Cleaning up disk space

`docker system df`
what uses space

`docker container prune`
remove stopped containers

`docker image prune`
remove dangling images

`docker image prune -a`
remove all unused images

`docker volume prune`
remove unused volumes

`docker builder prune`
clear the build cache

`docker network prune`
remove unused networks

`docker system prune`
stopped, dangling, unused networks

WATCH OUT

`docker system prune -a --volumes` also deletes unused images **and volumes**. Check `docker volume ls` first. A volume that no container uses right now may still hold your only copy of the data.

32 Docker Security in Depth

Layers of defence

Containers share the host's kernel, so careless images and settings are a real risk. **Layered habits** keep you safe.

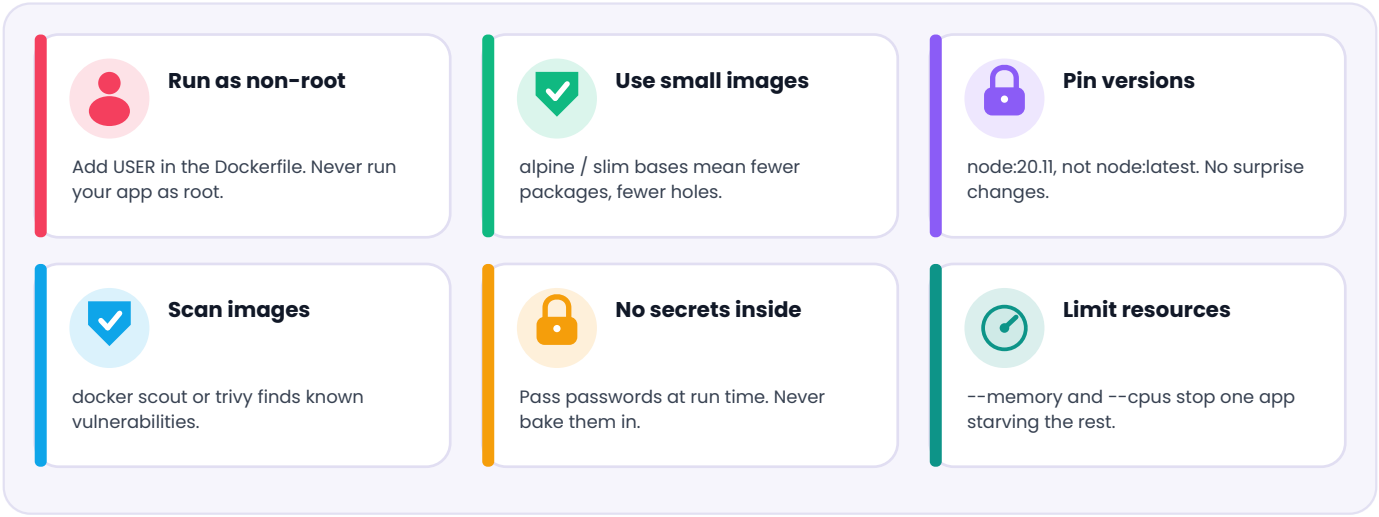


Fig 56. Six habits that make your containers much safer.

Defence in layers

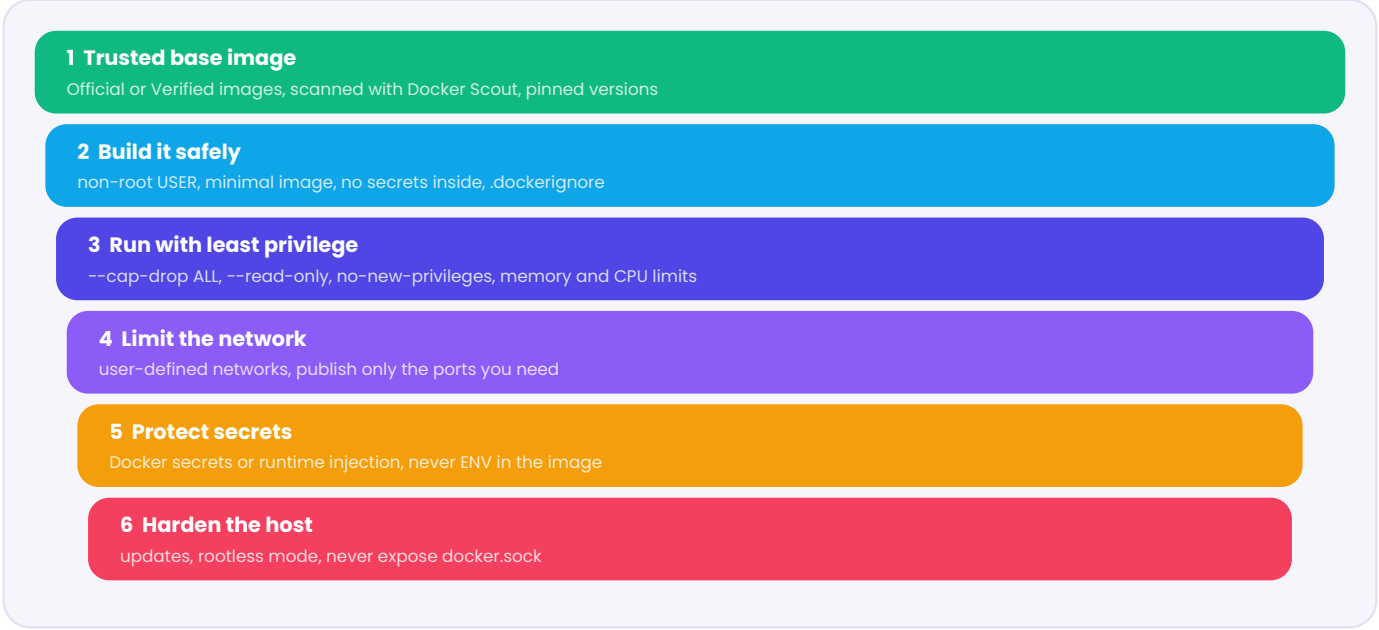


Fig 57. No single setting is enough. Each layer catches what the others miss.

A hardened docker run

```
terminal
docker run -d --name web \
  --user 10001:10001 --read-only --tmpfs /tmp \
  --cap-drop ALL --cap-add NET_BIND_SERVICE \
  --security-opt no-new-privileges:true \
  --memory 256m --cpus 0.5 --pids-limit 100 \
  -p 8080:8080 myapp:1.0
```

Option	What it protects against
`--user 10001:10001`	A bug running as root inside the container
`--read-only + --tmpfs /tmp`	Attackers writing files into the container
`--cap-drop ALL, --cap-add X`	Unneeded Linux superpowers. Add back only what the app truly needs
`--security-opt no-new-privileges`	Programs gaining extra rights through setuid files
`--memory, --cpus, --pids-limit`	One runaway container starving the host
`--network none or --internal network`	Containers that should never reach the internet

Dangerous settings to avoid

Do not	Why	Instead
`--privileged`	Gives the container almost every host power and device	Add only the one capability you need
`-v /var/run/docker.sock:/var/run/docker.sock`	Whoever controls it controls the host	Avoid it, or use a restricted proxy
`--security-opt seccomp=unconfined`	Turns off Docker's default system-call filter	Keep the default profile
`Secrets in ENV or image layers`	docker inspect and image history reveal them	Docker secrets or runtime injection
`Running as root`	A container escape becomes a root escape	USER in the Dockerfile

Rootless mode and user namespaces

Normally the Docker daemon runs as root. **Rootless mode** runs both the daemon and containers as a normal user, so a container escape does not give root. **User namespace remapping** makes root inside a container map to an unprivileged user on the host.

```
terminal
dockerd-rootless-setuptool.sh install              # set up the rootless daemon for your user
# or remap users in /etc/docker/daemon.json:
{ "userns-remap": "default" }
```

NOTE

Rootless mode has limits (for example, ports below 1024 and some network features). It is a great fit for development machines and shared servers.

Trust what you run

- Start from **Official Images** or **Verified Publishers**, and pin a version or digest.
- Scan every image with **Docker Scout** and update base images regularly.
- Turn on **Docker Content Trust** to run only signed images: `export DOCKER_CONTENT_TRUST=1`.
- Keep Docker itself and the host operating system updated.
- Use **Docker secrets** (or BuildKit secret mounts) for passwords and keys.

KEY IDEA

Least privilege: give a container only the access and resources it truly needs, nothing "just in case".

33 Docker Swarm

Orchestration built into Docker

Docker Swarm turns several machines into **one Docker cluster**. You say what should run, and Swarm keeps it running. It is built into Docker, so there is nothing extra to install. (Kubernetes is the other popular orchestrator and has its own separate notes.)

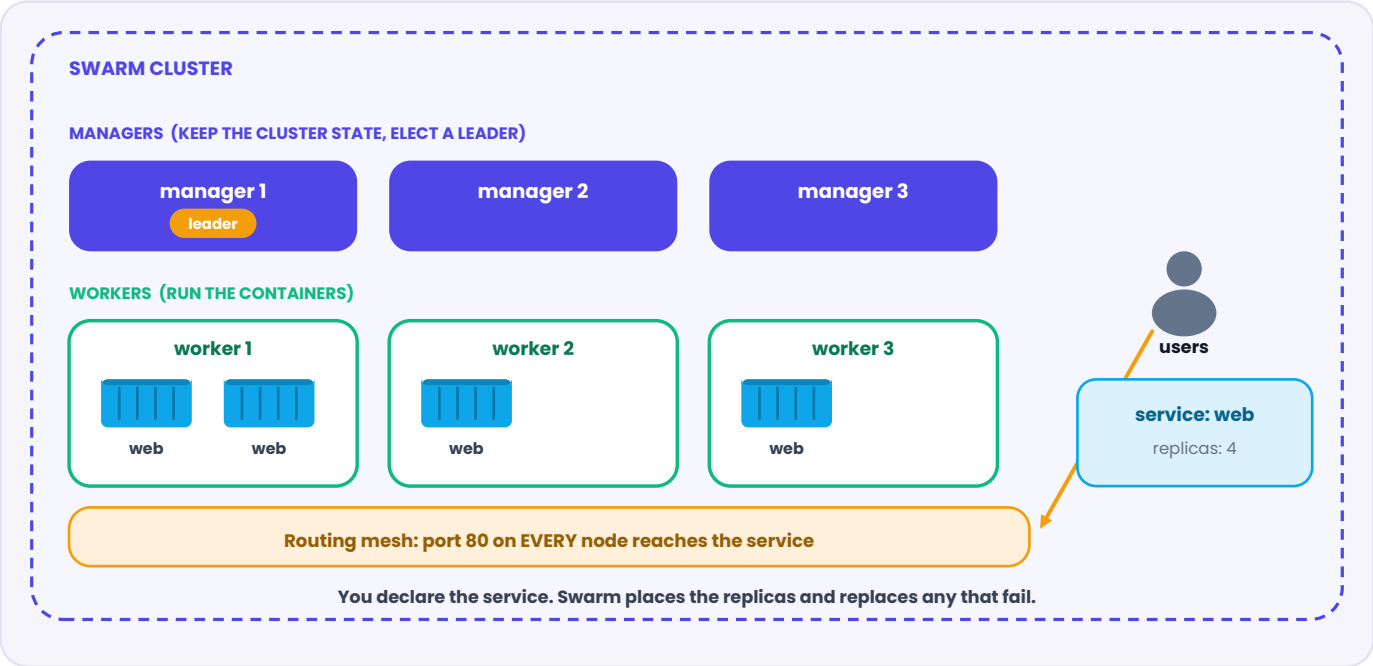


Fig 58. Managers keep the plan. Workers run the containers. The routing mesh sends traffic to the right place.

Words you need

Term	Meaning
Node	One machine in the cluster
Manager	A node that holds cluster state and gives orders. Use 3 or 5 for safety
Worker	A node that just runs containers
Service	What you want running: an image, ports, number of copies
Task / replica	One running container that belongs to a service
Replicated vs global	Replicated = N copies. Global = one copy on every node
Stack	A group of services deployed from one Compose file
Routing mesh	A published port works on every node and reaches a healthy replica

Build a cluster

```
terminal
docker swarm init --advertise-addr 192.168.1.10          # this machine becomes a manager
docker swarm join --token SWMTKN-... 192.168.1.10:2377      # run on each worker (init prints it)
docker node ls                                           # list the nodes
docker swarm leave                                       # a node leaves the cluster
```

Port	Used for
2377 / tcp	Cluster management
7946 / tcp and udp	Nodes talking to each other
4789 / udp	Overlay network traffic

Run and manage services

docker service create
start a service

docker service ls
list services

docker service ps web
where are its tasks?

docker service logs web
read its logs

docker service scale web=5
change the copies

docker secret create n file
store a secret (appears in /run/secrets)

```
terminal
docker service create --name web --replicas 3 -p 80:80 nginx:1.26
docker service update --image nginx:1.27 web         # rolling update, one replica at a time
docker service rollback web                          # go back to the previous version
```

Deploy a whole stack from a Compose file

```
compose.yaml
services:
  web:
    image: nginx:1.27
    ports: ["80:80"]
    deploy:                          # only used by Swarm
      replicas: 3
      update_config: { parallelism: 1, delay: 10s }    # update one replica every 10 seconds
      restart_policy: { condition: on-failure }
      resources:
        limits: { cpus: "0.5", memory: 128M }
```

```
terminal
docker stack deploy -c compose.yaml shop     # create or update the stack
docker stack services shop                      # see its services
docker stack rm shop                            # remove it
```

34 Practical: Dockerize an App

From code to container in five steps

Let us put everything together with a tiny Node.js app.

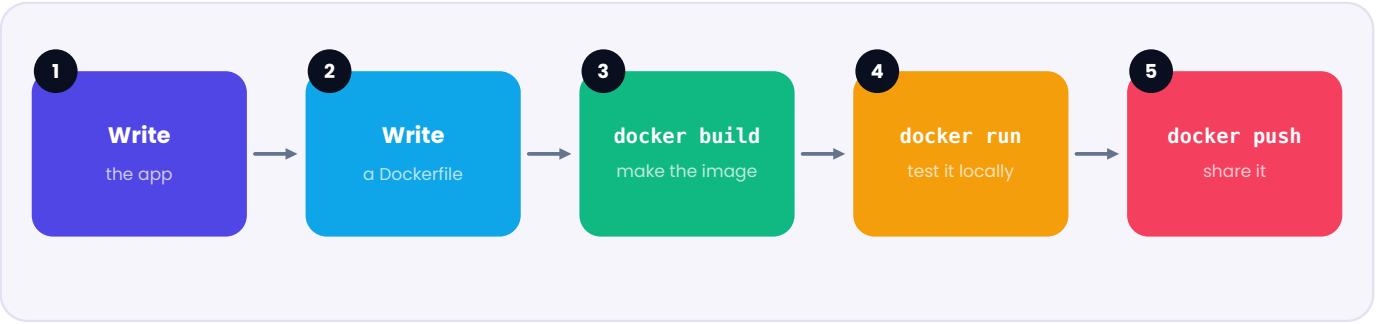


Fig 59. The whole journey in five steps.

Step 1 and 2: the app and its Dockerfile

server.js

```
// server.js
const express = require("express");
const app = express();
app.get("/", (req, res) => res.send("Hello from Docker!"));
app.listen(3000, () => console.log("Running on port 3000"));
```

package.json

```
{ "name": "hello-docker", "dependencies": { "express": "^4.19.0" } }
```

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

Step 3 and 4: build, run, test

terminal

```
docker build -t hello-docker .
docker run -d -p 3000:3000 --name hello hello-docker
docker logs hello
```

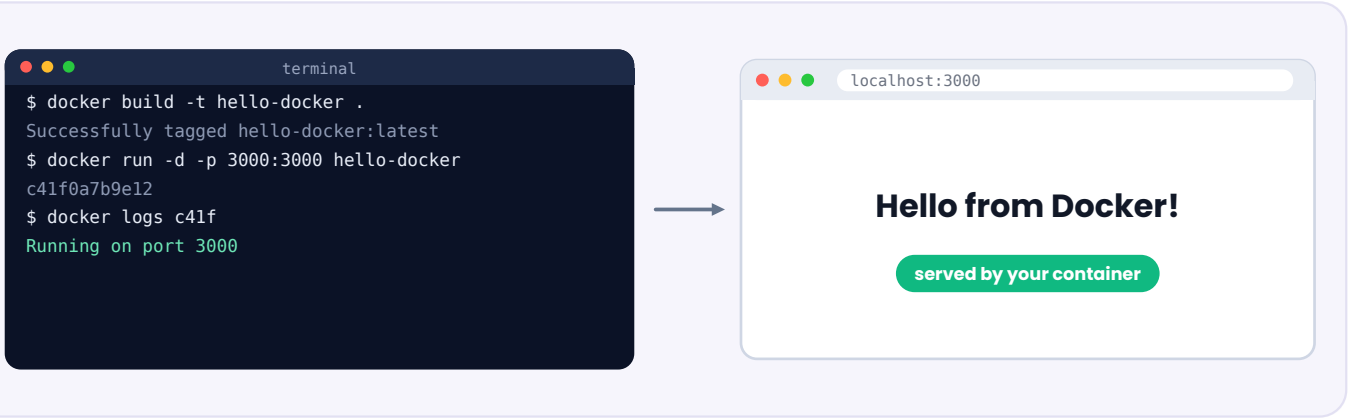


Fig 60. Build, run, and the page is served by your container.

Step 5: share it

```
terminal
docker tag hello-docker sharad/hello-docker:1.0
docker push sharad/hello-docker:1.0
```

TRY IT

Pull and run it from **another computer**: `docker run -p 3000:3000 sharad/hello-docker:1.0`. Same result, nothing to install. That is Docker's promise.

What you just used

Concept	Where it showed up
Image	Built from the Dockerfile with `docker build`
Container	Started with `docker run` and checked with `docker logs`
Port mapping	`-p 3000:3000` let your browser reach the app
Registry	`docker push` shared the image on Docker Hub

35 Practical: A Multi-container App

Flask and Redis with Compose

A two-container app: a **Flask** web page that counts visits using **Redis**. It uses a Dockerfile, Compose, a network, a volume and an environment variable.

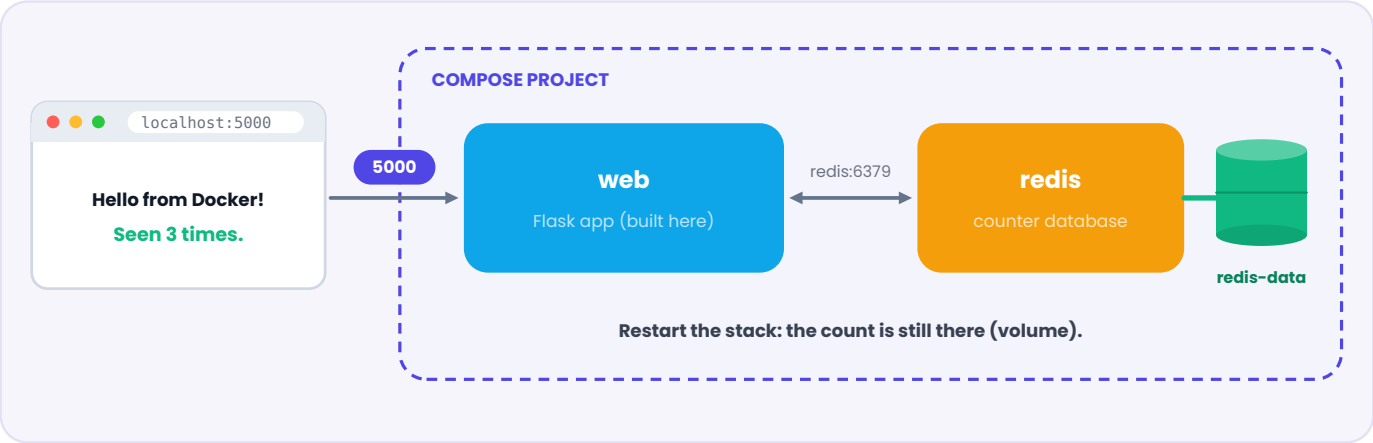


Fig 61. The web container talks to redis by name. A volume keeps the counter safe.

app.py

```
# app.py
import os
import redis
from flask import Flask

app = Flask(__name__)
r = redis.Redis(host=os.getenv("REDIS_HOST", "redis"), port=6379)

@app.route("/")
def hello():
    count = r.incr("hits")
    return f"Hello from Docker! Seen {count} times.\n"
```

requirements.txt

```
flask
redis
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
USER nobody
CMD ["flask", "--app", "app", "run", "--host=0.0.0.0"]
```

compose.yaml

```yaml
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      REDIS_HOST: redis
    depends_on:
      redis:
        condition: service_healthy
  redis:
    image: redis:7-alpine
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
volumes:
  redis-data:
```

Run it

terminal

```
docker compose up -d --build        # build the web image, start both services
curl localhost:5000                 # Hello from Docker! Seen 1 times.
curl localhost:5000                 # Seen 2 times.
docker compose ps                   # both services, redis shows (healthy)
docker compose logs -f web          # watch the requests arrive
```

Prove the volume works

terminal

```
docker compose down                 # remove the containers (the volume stays)
docker compose up -d
curl localhost:5000                 # the count continues, nothing was lost
docker compose down -v              # now delete the volume too: the counter resets
```

What you just used

Concept	Where it showed up
Dockerfile and image	The web image, built from python:3.12-slim
Compose	Two services started with one command
Networking	web reaches redis by the name redis
Volume	redis-data keeps the counter after the container is removed
Healthcheck and depends_on	web waits until redis answers ping
Environment variable	REDIS_HOST tells the app where the database is

36 Cheat Sheet and Glossary

Every command at a glance, plus key terms

Images	
`docker pull x`	download
`docker build -t n .`	build
`docker images`	list
`docker rmi x`	delete
`docker tag a b`	rename
`docker save / load`	to a file

Containers	
`docker run x`	create + start
`docker ps -a`	list all
`docker stop / start c`	stop / start
`docker rm c`	delete
`docker logs -f c`	follow logs
`docker exec -it c sh`	shell inside
`docker inspect c`	details

run options	
`-d -it --rm`	background, terminal, auto-delete
`--name -p -P`	name, publish ports
`-v --mount --tmpfs`	storage
`-e --env-file`	variables
`--restart --memory --cpus`	behaviour, limits
`--user --read-only`	security

Dockerfile	
`FROM WORKDIR`	base, folder
`COPY RUN`	add files, run commands
`ENV ARG`	variables
`EXPOSE VOLUME`	port, data folder
`USER HEALTHCHECK`	user, health test
`CMD ENTRYPOINT`	start command

Networking	
`docker network create n`	new network
`docker network ls`	list
`docker network connect n c`	attach
`--network n`	join at run
`-p 8080:80`	publish port

Compose	
`docker compose up -d`	start all
`docker compose down`	stop all
`docker compose ps`	list
`docker compose logs -f`	follow logs
`docker compose build`	rebuild
`docker compose exec s sh`	shell
`docker compose config`	check file

Registries	
`docker login [host]`	sign in
`docker push user/app:1.0`	upload
`docker pull user/app:1.0`	download
`docker logout`	sign out
`ghcr.io/owner/app`	GitHub registry name

Scout and init	
`docker scout quickview`	summary
`docker scout cves`	vulnerabilities
`docker scout recommendations`	base image advice
`docker init`	generate Docker files

Volumes	
`docker volume create v`	new volume
`docker volume ls / rm`	list / delete
`-v name:/path`	mount volume
`-v $(pwd):/path`	bind mount

Debug and clean	
`docker system df`	disk usage
`docker stats`	live usage
`docker events`	live events
`docker system prune`	clean up
`docker builder prune`	build cache

Swarm	
`docker swarm init`	make a cluster
`docker service create`	run a service
`docker service scale s=5`	change copies
`docker stack deploy -c f s`	deploy a stack

Glossary

Image	Read-only template used to create containers	**Container**	A running (or stopped) instance of an image
Dockerfile	Text file of build instructions for an image	**Layer**	One cached, read-only change inside an image
Registry	Server that stores and shares images	**Docker Hub**	Docker's default public registry
GHCR	GitHub Container Registry, ghcr.io	**Tag**	Movable label on an image, usually a version
Digest	Fixed fingerprint (sha256) of exact image content	**Volume**	Docker-managed storage that outlives containers
Bind mount	A host folder mapped into a container	**tmpfs**	Memory-only storage that vanishes on stop
Bridge network	Default private network on one host	**Overlay network**	Network that spans several Docker hosts
macvlan	Gives a container its own address on your LAN	**Daemon**	dockerd, the background service doing the work
containerd	Runtime that pulls images and manages containers	**runc**	Low-level tool that creates the container
OCI	Open standards for images and runtimes	**Namespace**	Linux feature giving a container its own view
cgroup	Linux feature limiting CPU, memory and I/O	**Compose**	Runs multi-container apps from one YAML file
Service	One container definition in Compose or Swarm	**Healthcheck**	A command Docker runs to test the app
Multi-stage build	Dockerfile with several stages, ships only the last	**BuildKit**	Docker's modern image builder
buildx	CLI for BuildKit and multi-platform builds	**Docker Scout**	Image vulnerability analysis
SBOM	Software bill of materials: list of packages inside	**CVE**	Public ID of a known vulnerability
Swarm	Docker's built-in cluster mode	**Context**	Which Docker host the CLI talks to
Rootless mode	Docker daemon and containers without root	**Registry mirror**	Cache that serves images instead of the original registry

TRY IT

Dockerize one small app you already have this week. One real rep beats rereading these notes.